



OPINION ARTICLE

REVISED **An environment for sustainable research software in Germany and beyond: current state, open challenges, and call for action**

[version 2; peer review: 2 approved]

Hartwig Anzt^{1,2*}, Felix Bach ^{1*}, Stephan Druskat ^{3-5*}, Frank Löffler ^{3,6*}, Axel Loewe ^{1*}, Bernhard Y. Renard^{7*}, Gunnar Seemann ^{8*}, Alexander Struck ^{5*}, Elke Achhammer⁹, Piush Aggarwal ¹⁰, Franziska Appel¹¹, Michael Bader⁹, Lutz Brusch ¹², Christian Busse ¹³, Gerasimos Chourdakis ⁹, Piotr Wojciech Dabrowski ¹⁴, Peter Ebert¹⁵, Bernd Flemisch¹⁶, Sven Friedl ¹⁷, Bernadette Fritsch ¹⁸, Maximilian D. Funk¹⁹, Volker Gast³, Florian Goth²⁰, Jean-Noël Grad ¹⁶, Jan Hegewald ¹⁸, Sibylle Hermann¹⁶, Florian Hohmann²¹, Stephan Janosch²², Dominik Kutra ²³, Jan Linxweiler ²⁴, Thilo Muth ²⁵, Wolfgang Peters-Kottig ²⁶, Fabian Rack²⁷, Fabian H.C. Raters ²⁸, Stephan Rave ²⁹, Guido Reina ¹⁶, Malte Reißig ³⁰, Timo Ropinski^{31,32}, Joerg Schaarschmidt¹, Heidi Seibold ³³, Jan P. Thiele ³⁴, Benjamin Uekermann ³⁵, Stefan Unger³⁶, Rudolf Weeber¹⁶

¹Karlsruhe Institute of Technology (KIT), Karlsruhe, Germany

²University of Tennessee, Knoxville, TN, USA

³Friedrich Schiller University, Jena, Germany

⁴German Aerospace Center (DLR), Berlin, Germany

⁵Humboldt-Universität zu Berlin, Berlin, Germany

⁶Louisiana State University, Baton Rouge, LA, USA

⁷Hasso Plattner Institute, Potsdam, Germany

⁸University Heart Centre Freiburg Bad Krozingen, Freiburg, Germany

⁹Technische Universität München, München, Germany

¹⁰Universität Duisburg-Essen, Duisburg, Germany

¹¹Leibniz Institute of Agricultural Development in Transition Economies (IAMO), Halle (Saale), Germany

¹²Technische Universität Dresden, Dresden, Germany

¹³Deutsches Krebsforschungszentrum, Heidelberg, Germany

¹⁴Hochschule für Technik und Wirtschaft Berlin, Berlin, Germany

¹⁵Saarland Informatics Campus, Saarbrücken, Germany

¹⁶University of Stuttgart, Stuttgart, Germany

¹⁷Berlin Institute of Health, Berlin, Germany

¹⁸Alfred Wegener Institute, Bremerhaven, Germany

¹⁹Max-Planck-Gesellschaft e.V., München, Germany

²⁰Universität Würzburg, Würzburg, Germany

²¹Universität Bremen, Bremen, Germany

²²Max Planck Institute of Molecular Cell Biology and Genetics, Dresden, Germany

²³European Molecular Biology Laboratory, Heidelberg, Germany

²⁴Technische Universität Braunschweig, Braunschweig, Germany

²⁵Federal Institute for Materials Research and Testing, Berlin, Germany

- ²⁶Konrad-Zuse-Zentrum für Informationstechnik Berlin (ZIB), Berlin, Germany
²⁷FIZ Karlsruhe - Leibniz Institute for Information Infrastructure, Karlsruhe, Germany
²⁸University of Goettingen, Göttingen, Germany
²⁹University of Münster, Münster, Germany
³⁰Institute for Advanced Sustainability Studies, Potsdam, Germany
³¹Ulm University, Ulm, Germany
³²Linköping University, Linköping, Sweden
³³Ludwig Maximilian University of Munich, München, Germany
³⁴Leibniz University Hannover, Hannover, Germany
³⁵Eindhoven University of Technology, Eindhoven, The Netherlands
³⁶Julius Kühn-Institut (JKI), Quedlinburg, Germany

* Equal contributors

v2 First published: 27 Apr 2020, 9:295
<https://doi.org/10.12688/f1000research.23224.1>
 Latest published: 26 Jan 2021, 9:295
<https://doi.org/10.12688/f1000research.23224.2>

Abstract

Research software has become a central asset in academic research. It optimizes existing and enables new research methods, implements and embeds research knowledge, and constitutes an essential research product in itself. Research software must be sustainable in order to understand, replicate, reproduce, and build upon existing research or conduct new research effectively. In other words, software must be available, discoverable, usable, and adaptable to new needs, both now and in the future. Research software therefore requires an environment that supports sustainability.

Hence, a change is needed in the way research software development and maintenance are currently motivated, incentivized, funded, structurally and infrastructurally supported, and legally treated. Failing to do so will threaten the quality and validity of research. In this paper, we identify challenges for research software sustainability in Germany and beyond, in terms of motivation, selection, research software engineering personnel, funding, infrastructure, and legal aspects. Besides researchers, we specifically address political and academic decision-makers to increase awareness of the importance and needs of sustainable research software practices. In particular, we recommend strategies and measures to create an environment for sustainable research software, with the ultimate goal to ensure that software-driven research is valid, reproducible and sustainable, and that software is recognized as a first class citizen in research. This paper is the outcome of two workshops run in Germany in 2019, at deRSE19 - the first International Conference of Research Software Engineers in Germany - and a dedicated DFG-supported follow-up workshop in Berlin.

Keywords

Sustainable Software Development, Academic Software, Software Infrastructure, Software Training, Software Licensing, Research Software

Open Peer Review

Approval Status  

	1	2
version 2 (revision) 26 Jan 2021	 view	
version 1 27 Apr 2020	 view	 view
1. Wilhelm Hasselbring , Kiel University, Kiel, Germany 2. Radovan Bast , UiT The Arctic University of Norway, Tromsø, Norway		
Any reports and responses or comments on the article can be found at the end of the article.		



This article is included in the **Research on Research, Policy & Culture gateway**.



This article is included in the **Max Planck Society collection**.

Corresponding authors: Axel Loewe (axel.loewe@kit.edu), Gunnar Seemann (gunnar.seemann@universitaets-herzzentrum.de)

Author roles: **Anzt H:** Conceptualization, Writing – Original Draft Preparation, Writing – Review & Editing; **Bach F:** Conceptualization, Investigation, Writing – Original Draft Preparation, Writing – Review & Editing; **Druskat S:** Conceptualization, Investigation, Writing – Original Draft Preparation, Writing – Review & Editing; **Löffler F:** Conceptualization, Investigation, Visualization, Writing – Original Draft Preparation, Writing – Review & Editing; **Loewe A:** Conceptualization, Funding Acquisition, Investigation, Project Administration, Writing – Original Draft Preparation, Writing – Review & Editing; **Renard BY:** Conceptualization, Investigation, Writing – Original Draft Preparation, Writing – Review & Editing; **Seemann G:** Conceptualization, Funding Acquisition, Investigation, Project Administration, Writing – Original Draft Preparation, Writing – Review & Editing; **Struck A:** Conceptualization, Writing – Original Draft Preparation, Writing – Review & Editing; **Achhammer E:** Writing – Original Draft Preparation; **Aggarwal P:** Writing – Original Draft Preparation; **Appel F:** Writing – Original Draft Preparation, Writing – Review & Editing; **Bader M:** Writing – Original Draft Preparation, Writing – Review & Editing; **Brusch L:** Writing – Original Draft Preparation, Writing – Review & Editing; **Busse C:** Writing – Review & Editing; **Chourdakis G:** Writing – Review & Editing; **Dabrowski PW:** Writing – Review & Editing; **Ebert P:** Writing – Original Draft Preparation; **Flemisch B:** Writing – Original Draft Preparation; **Friedl S:** Visualization, Writing – Original Draft Preparation, Writing – Review & Editing; **Fritzsche B:** Writing – Review & Editing; **Funk MD:** Writing – Original Draft Preparation, Writing – Review & Editing; **Gast V:** Writing – Review & Editing; **Goth F:** Writing – Original Draft Preparation; **Grad JN:** Writing – Original Draft Preparation; **Hegewald J:** Writing – Review & Editing; **Hermann S:** Writing – Original Draft Preparation, Writing – Review & Editing; **Hohmann F:** Writing – Original Draft Preparation; **Janosch S:** Writing – Review & Editing; **Kutra D:** Writing – Original Draft Preparation; **Linxweiler J:** Writing – Original Draft Preparation; **Muth T:** Writing – Original Draft Preparation; **Peters-Kottig W:** Writing – Original Draft Preparation; **Rack F:** Writing – Original Draft Preparation, Writing – Review & Editing; **Raters FHC:** Writing – Original Draft Preparation; **Rave S:** Writing – Original Draft Preparation; **Reina G:** Writing – Original Draft Preparation, Writing – Review & Editing; **Reißig M:** Writing – Review & Editing; **Ropinski T:** Writing – Original Draft Preparation; **Schaarschmidt J:** Writing – Original Draft Preparation; **Seibold H:** Writing – Review & Editing; **Thiele JP:** Writing – Original Draft Preparation, Writing – Review & Editing; **Uekermann B:** Writing – Original Draft Preparation, Writing – Review & Editing; **Unger S:** Visualization, Writing – Original Draft Preparation; **Weeber R:** Writing – Original Draft Preparation

Competing interests: No competing interests were disclosed.

Grant information: The authors thank the Deutsche Forschungsgemeinschaft (DFG) for funding a meeting (Rundgespräch, grants LO 2093/3-1 and SE 1758/6-1) during which the initial draft of this paper has been created. We are particularly grateful for the support from Dr. Matthias Katerbow (DFG). This work was additionally supported by Research Software Sustainability grants funded by the DFG: Aggarwal: 390886566; PI: Zesch. Appel: 391099391; PI: Balmann. Bach & Loewe & Seemann: 391128822; PIs: Loewe/Scholze/Seemann/Selzer/Streit/Upmeier. Bader: 391134334; PIs: Bader/Gabriel/Frank. Brusch: 391070520; PI: Brusch. Druskat & Gast: 391160252; PI: Gast/Lüdeling. Ebert: 391137747; PI: Marschall. Flemisch & Hermann: 391049448; PIs: Boehringer/Flemisch/Hermann. Hohmann: 391054082; PI: Hepp. Goth: 390966303; PI: Assaad. Grad & Weeber: 391126171; PI: Holm. Kutra: 391125810; PI: Kreshuk. Mehl & Uekermann: 391150578; PIs: Bungartz/Mehl/Uekermann. Peters-Kottig: 391087700; PIs: Gleixner/Peters-Kottig/Shinano/Sperber. Raters: 39099699; PI: Herwartz. Reina: 391302154; PIs: Ertl/Reina. Muth & Renard: 391179955; PIs: Renard/Fuchs. Ropinski: 391107954; PI: Ropinski. Alexander Struck acknowledges the support of the Cluster of Excellence Matters of Activity. Image Space Material funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany's Excellence Strategy – EXC 2025. We acknowledge support by the KIT-Publication Fund of the Karlsruhe Institute of Technology. *The funders had no role in study design, data collection and analysis, decision to publish, or preparation of the manuscript.*

Copyright: © 2021 Anzt H *et al.* This is an open access article distributed under the terms of the **Creative Commons Attribution License**, which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited.

How to cite this article: Anzt H, Bach F, Druskat S *et al.* **An environment for sustainable research software in Germany and beyond: current state, open challenges, and call for action [version 2; peer review: 2 approved]** F1000Research 2021, 9:295 <https://doi.org/10.12688/f1000research.23224.2>

First published: 27 Apr 2020, 9:295 <https://doi.org/10.12688/f1000research.23224.1>

REVISED Amendments from Version 1

Besides fixing some typographic errors and adding references as suggested by the reviewers,

We separated the legal decision tress from this manuscript. As they were not the focus of this position paper and diluted its key messages, they were published separately: <https://doi.org/10.5281/zenodo.4327147>

Other aspects that were elaborated on include: testing, infrastructure for cross-institutional use, sustainable funding, the relation between software quality and transparency, a clear statement pro open source, the potential role of legal help desks.

Jan Hegewald was added to the list of authors. He already contributed to initial submission but unfortunately his name was missing in the list.

Any further responses from the reviewers can be found at the end of the article

Background

Meet Kim, who is currently a post-grad PhD student in researchonomy at the University of Arcadia (UoFA). We will follow Kim's fictional career in order to understand different aspects of research software sustainability. Note that in Kim's world, many of the changes this paper calls for have already been implemented. (In our example, Kim is a female person. Of course, research software engineers (RSEs) can be of any gender.)

Computational analysis of large data sets, computer-based simulations, and software technology in general play a central role for virtually all scientific breakthroughs of at least the 21st century. The first image of a black hole may be the most prominent recent example where astrophysical experiments and the collection and processing of data had to be complemented with sophisticated algorithms and software to enable research excellence^{1,2}. Similarly, it is research software that allows us to get a glimpse of the consequences our actions today have on the climate of tomorrow. However, an implication of computer-based research is that findings and data can only be reproduced, understood, and validated if the software that was used in the research process is sustained and their functionality maintained.

At the same time, sustaining research software, and in particular open research software, comes with a number of challenges. Commercial research software often has revenue flows that can facilitate sustainable software development, maintenance, and documentation as well as the operation of adequate infrastructure. However, a large share of researchers base their research on software that was developed in-house or as a community effort. Many of these software stacks can not be sustained – often because research software was not a first class deliverable in a research project and hence remained in a prototype state, or because of missing incentives and resources to maintain the software after project funding ended. Another fundamental difference to industrial software development is that most developers of academic research software (often doctoral students or postdoctoral researchers) never receive training in sustainable software development³. In particular, as they see themselves usually as the primary user of a software product, there are virtually no incentives to invest in sustainability measures such as code documentation or portability.

In combination with the predominance of temporary positions in research, this results in a highly inefficient system where millions of lines of code are generated every year that will not be re-used after the termination of the developer's position. Part of the problem is the reluctance to accept research software engineering as an academic profession that results in a lack of incentives to produce high-quality software: producing high software quality needs sufficient resources, and although the [San Francisco Declaration on Research Assessment \(DORA\)](#) demands a change in the academic credit system, many institutions base promotion and appointments on traditional metrics like the Hirsch index⁴. It is obvious that an extraordinary amount of idealism is required to write sustainable code, including documentation and installation routines, as well as running infrastructure and giving support to others when resources can be used more profitably in writing scientific publications based on fragile prototype software^{5,6}.

Thus, one main factor for the poor sustainability of research software is the lack of long-term funding for research software engineers (RSEs)^{7,8} who take care of the appropriate architecture, organization, implementation, documentation, and community interaction for the software, paired with the implementation of measures towards making the software sustainable during and beyond the development process⁹.

In this paper, we describe the state of the practice and current challenges for research software sustainability and suggest measures towards improvements that can solve these challenges. The paper is the result of a community effort, with work undertaken during two workshops and subsequent collaborative work across the larger RSE community in Germany. It has been initiated during a half-day workshop at [first International Conference for Research Software Engineers in Germany \(deRSE19\)](#) in Potsdam, Germany on June 5th, 2019, and continued during a dedicated two-day workshop in Berlin, Germany on November 7th and 8th, 2019, which was funded by the German Research Foundation (Deutsche Forschungsgemeinschaft, DFG). Subsequently, the draft produced during the latter event was opened up for collaborative discussion by the German RSE community through [de-RSE e.V. - Society for Research Software](#).

We mainly focus on the situation of research software and RSEs in Germany, where funding bodies increasingly acknowledge the importance and value of sustainable research software and related infrastructures. The DFG, the largest funding body for fundamental research in Germany, for example, opened a [call for sustainable research software development](#) at the end of 2016 and a second [call for quality management in research software](#) in June 2019. The first call was oversubscribed by a factor of 10-15, a strong indicator of unmet demand. As another example, the 2019 [“Guidelines for Safeguarding Good Research Practice” codex of the DFG](#) now explicitly lists software side-by-side with other research results and data. The FAIR principles for research data¹⁰ provide guidelines for data archiving, but enabling full reproducibility and traceability of research software requires additional steps¹¹. In consequence, there are ongoing discussions on whether software should be considered as a specific kind of research data or as a separate entity¹².

These positive developments notwithstanding, guidelines and policies for sustainable research software development in

Germany are unfortunately still lacking, and long-term funding strategies are missing. This all leads to unmet requirements and unsolved challenges that we want to highlight in this paper by elaborating on (1) why research software engineering needs to be considered an integral part of academic research; (2) how to decide which software to sustain; (3) who sustains research software; (4) how software can be funded sustainably; (5) what infrastructure is needed for sustainable software development; and (6) legal aspects of research software development in academia. While we specifically focus on the research software landscape in Germany, we are convinced that many of the analyses, findings, and recommendations may carry beyond. We want to address RSEs who are experiencing similar challenges and newcomers to the field of research software development, but first and foremost political and academic decision makers to raise awareness of the importance of and requirements for sustainable software development. As a community, we work hard on overcoming the challenges of software development in an academic setting, but we need support – and reliable funding options and institutional recognition in particular – for the sake of better research.

Why sustainable research software in the first place?

After graduation, Kim joins a fixed-term researchonomical research project. For her PhD thesis, she wants to crunch some data. Her colleague recommends learning some Boa, which is an all-purpose programming language often used in researchonomy. Luckily, the UoFA runs regular Software Plumbery courses for researchers, including a Boa course. Kim takes the course and gains a solid understanding of the basics of the Hash shell, version control with Tig, and the basics of Boa. She starts writing scripts, which help her a lot with the data processing. Unfortunately, Kim's scripts are quite slow and actually break after she installs a newer version of Boa. She visits the weekly Code Café organized by her university's central RSE team. The RSEs not only help her update her scripts but also suggest some changes which speed up the computation by a factor of 25.

During the next meeting with her PhD supervisor, Kim presents her collection of scripts. The supervisor encourages Kim to create a Boa library from them, as they will be very useful to other researchonomists. Thankfully, Kim's project PI had applied for three RSE person months in their grant, so the project enlists an RSE from the central team. Over the next three months, Kim and the RSE work together to build the library, document it, test it, license it under the permissive Comanche license, update the TigLab repository to let others contribute, introduce automated builds for every code change via a continuous integration platform, and make the library citable. Finally, they release the first major version of the library, named *hal9k* and publish it through the university library's software portal, where they get a DOI (Digital Object Identifier) for the version as well as a concept DOI for any future versions of the library. Working with the RSE, Kim has gained a good understanding of some methods in software engineering, and she's thrilled because this also means she'll be able to get a job with a local tech company once her fixed-term contract has run out.

Kim passes her PhD - of which *hal9k* is an important part - with flying colors, and soon citations to her library start appearing in the researchonomic literature. To Kim's surprise, she also reads a blog post about a citizen science maker project which has used *hal9k* to process researchonomic data measured in a neighborhood of her hometown. She is invited to give a talk at the local office of Siren, a global tech company, which look to adopt *hal9k*, and pay Kim a generous speaker honorarium. So generous in fact, that Kim can pay a student assistant for a full year from the money.

Our credibility as researchers in society hinges on the notion of proper research conduct, also known as “good research practice”. The digitalization of research has introduced complex digital research outputs, such as software and data sets. Although first recommendations¹³ and policies¹⁴ exist, they are far from being widely adopted. It is still somewhat unclear how to translate good research practice into good research software practice, for example in terms of validity and reproducibility, but also pertaining to the responsible use of resources. The damage that failing to do so is causing both to the progress of the research community and to the credibility of academic research in society is becoming increasingly clear with the growth of the replication crisis - while the lack of universally agreed-upon and supported good research software practice is not the main reason for that crisis, it clearly is a contributing factor.

While it is obvious that software qualifies as a potentially re-usable digital artifact, the additional benefit of not just reproducing a given scenario, but transferring software use to new problems, domains, and/or applications, justifies developing research software with a long-term perspective as *sustainable research software*.

In order to support research, a sustainable software must ideally be correct^{15–17}, and validatable. Due to the experimental nature of some research software, this may not be possible in all cases, e.g., due to lack, or infeasible implementation, of a test oracle¹⁸, vast configuration spaces, or large and heterogeneous data inputs¹⁹. While it must be accepted that precise, oracle-based testing may not be possible here, alternative solutions should be implemented, such as metamorphic testing, runtime assertions, test input sampling and generation (e.g. via machine learning), and input data modeling. Sustainable software must also be understandable, documented, publicly released, adequately published (i.e. in persistently identifiable form as software source code²⁰, and potentially in an additional paper which describes the software concept, design decisions, and development rationale), actively maintained, and (re-)usable^{21–23}. We also argue that truly sustainable research software should ideally be published under a Free/Libre Open Source Software (FLOSS) license, and follow an open development model, to (1) enable the validation of research results that have been produced using the software, (2) enable the reproducibility of software-based research, (3) enable improvement and (re-) use of the software to support more and better research, and reduce resources to be spent on software development, (4) reduce legal issues (see section below), (5) meet ethical obligations from public funding, and (6) open research software to the general public, i.e., the stakeholder group with arguably the greatest interest in furthering research knowledge and improving research for the benefit of all.

To make software-based research (and with that almost any research) reproducible, the used software must continue to exist. Furthermore, it must continue to be usable, understandable, and return consistent results (or potential changes to results and bug fixes must be clearly documented) in the evolving software and hardware environment. Moreover, the software should support reuse scenarios to avoid duplication of efforts and drain of resources. Therefore, if research software is publicly funded, it should be freely available under a FLOSS license.

Currently, creating and using sustainable research software is not sufficiently incentivized. To evaluate in which area this shortcoming should be addressed, we have identified the following challenges:

- **Lack of benefit for the individual:** Currently, the primary motivation for sustainable research software is the common benefit, rather than the individual benefit. It is clearly beneficial for the research community as a whole to direct resources towards sustainable research software, as it enables better and more research by freeing funds for domain research rather than (repetitive) software development. But the developers are often even at a disadvantage (e.g., they publish fewer papers^{5,6}), which in turn prevents sustainable research software.
 - **Lack of suitable incentive systems:** Contributions to research that are not traditional text-based products (i.e., papers or monographs) are still not sufficiently rewarded, or not rewarded at all, due to the missing implementation of mandatory software citation^{20,24-32}, among other reasons. Interestingly, one third of research software repositories have a lifespan (defined as the time from the first time any code was uploaded to the last contribution) of less than one day (median: 15 days¹¹), indicating that many codes are only made available publicly for the publication in a journal (as increasingly encouraged or required by journals³³ and associated with higher impact³⁴) but are not maintained thereafter.
 - **Lack of awareness:** Research software sustainability and its importance is lacking visibility as well as acceptance³⁵⁻³⁸, and research software engineering in its implementation as sustainable software development and software maintenance is not sufficiently supported, both in Germany and beyond^{9,39,40}.
 - **Lack of expertise:** Knowledge about how to create, maintain, and support sustainable research software is emerging⁴¹⁻⁴³ but has not yet permeated related activities within organizations - specifically teaching, mentoring, and consultancy. This lack of expertise can also lead to divergence between software design and community uptake, e.g., if the software fails to meet the needs of the target group, or is insufficiently usable. RSEs combine sustainable software engineering expertise with experience in one or more research domains.
 - **Heterogeneous research community:** There are significant differences with respect to how software is developed, published, used, and valued in the different academic disciplines. Additionally, there is even heterogeneity within a community in terms of application and approach. This also makes it hard to train researchers for sustainable software development, as beyond basic training in computational research such as provided by [The Carpentries](#), advanced courses for research software engineering are not widely available (with the notable exception of the [CodeRefinery project](#)). Targeted curricula must be developed and updated regularly, and specialized instructors need to be trained.
 - **Lack of impact measures:** It is unclear how to measure the impact of research software with respect to its quality, reusability, and benefit for the research community. This exceeds the implementation of research software citation (which is work in progress^{20,31,32,44}), and pertains to sustainability and policy studies.
 - **Infrastructure issues:** Due to a lack of knowledge about how sustainability features impact the application of research software, there is not yet enough evidence for whether centralized or decentralized facilities should be favored to further research software sustainability⁴⁵⁻⁴⁷. Commonly, local infrastructure hinders cross-institutional collaboration, whereas cross-organizational infrastructures often suffer from lack of authentication and authorization implementations, or legal constraints. This in turn leads to a lack of infrastructure as a whole.
 - **Legal issues:** Many obstacles for research software pertain to legal issues, such as applicable licensing and compatibility of licenses⁴⁸, and decisions about license types.
 - **Funding issues:** Despite some individual initiatives⁴⁹⁻⁵², funding for the creation, maintenance, and support of sustainable research software is still scarce. Additionally, existing models usually supply seed funding only, which disregards the support and maintenance steps in the software development lifecycle. Instead, a potential "market" is relied upon to support these, which may only develop long after the initial project has ended. With regard to the funding of infrastructure which underpins modern development approaches such as DevOps and continuous deployment, cloud infrastructure providers and their pricing models do not work well with current funding models, due to lack of knowledge of how to target them with traditional academic funding and budgeting, compliance issues, or rigid bureaucracy.
 - **Slow adoption of research software engineering as a profession:** Career options for research software work are not fully determined, although career paths are emerging in some regions. Initially, the RSE initiative in the UK has made progress in this area, and RSE groups have been installed in many institutions. In [Germany](#), the [US](#), and the [Netherlands](#), this is still work in progress. It is also not yet determined how to match research software engineering roles in public institutions with industry roles⁵³.
- In summary, the necessary but resource-intensive practice of creating, maintaining, supporting, and funding sustainable research software is not yet sufficiently incentivized and enabled by research institutions and funding agencies, nor does it align well with the publish-or-perish culture that is still prominent in most fields.
- Therefore, it is necessary to comprehensively motivate sustainable research software practice. In the following, we identify stakeholders of research software⁵⁴⁻⁵⁶, and explicate their particular motivations for sustainable research software.

Subsequently, we specify challenges towards satisfying the demands of the individual stakeholders.

Stakeholder motivations for research software sustainability

While a wide range of stakeholders share interest in sustainable software, we argue that their individual motivation can differ quite significantly:

The *general public* benefits from research which supports the common good, in other terms: creates a better world, faster. Taxpayers have an interest in economical use of their tax money, to which duplicated or flawed efforts to create research software – in contrast to software reuse – is contrary. A subset of this group may be interested in sustainable, i.e., re-usable and understandable, software as part of citizen science.

Domain researchers benefit from better software to do more, better, and faster research. Sustainable research software supports this through validated functionality (e.g., correct algorithms), the potential for reuse, and general availability. Sustainable software also potentially simplifies building upon previous research results by reusing the involved software to produce additional data or by extending the software's functionality. In light of recent updates to [definitions of good research practice](#), sustainable research software also allows domain researchers to comply with guidelines and best practices. Additionally, using software that is sustainable enough to establish itself as a standard tool in a field signifies inclusion in a research community. Less directly, researchers may benefit from the existence of sustainable standard tools as they yield standard formats, which in themselves facilitate reuse of research data.

Research software engineers (RSEs) have an intrinsic interest in sustainable research software. They create better software for research, which enables more and better research. RSEs have an inherent interest in developing and working with high quality software, as part of professional ethics as well as good research practice. RSEs build their reputation on high quality software and software citation^{20,31}, which will open up new career paths. Finally, for RSEs, creating sustainable research software is part of an attractive, intellectually challenging, and satisfying work environment.

Research leaders as well as *research performing organizations* mainly focus on the economic aspects and management of research, i.e., available funds, people, and time employed to optimize research output. Both need to make sure that their employees continually improve their qualification and generate impact to improve their standing in the various research communities and ensure continued funding. Overseeing and enabling the creation of sustainable research software advances their visibility in the field and makes their research endeavors both more future-proof and more easily traceable, reproducible, and verifiable and thus more likely to attract additional resources (including human resources). *Research performing organizations* can additionally benefit from sustainable research software if it can be reused in other areas, creating synergies between different research disciplines. These

synergies typically free resources that can then be used in areas other than software development and maintenance. Finally, organizations can gain highly competitive positions in terms of funding and hiring opportunities, as well as a reputation for being on the cutting edge of research, through early adoption of research software engineering units, and the implementation of sustainable research software policy and practice.

Research funding organizations have inherent interest in – and directly benefit from – the existence of sustainable research software as it allows them to direct more resources towards actual research (rather than recreation of software) and increase return on investment. At the same time, funding organizations can create incentives for sustainable software by imposing policies that reflect the necessity of research software sustainability and creating respective funding opportunities.

Geopolitical units have a strategic interest to be independent of other geopolitical units to ensure that research can continue seamlessly regardless of geopolitical developments and ensuing embargoes on information flow. Reuse of sustainable software additionally frees up funding for uses other than software development. Well-established, sustainable software systems can also attract researchers and companies in the research and technology sector.

Libraries (also registries, indices) benefit from sustainable research software, as it will undergo a formal publishing process and be properly described in its metadata. Libraries can extend their portfolio beyond text-based research objects and stake claims as organizations harnessing the digitalization of research. In turn, they help to increase visibility and discoverability for research software through their services and advance the competitiveness of their organization or geopolitical unit. In addition, libraries also use research software and would thus benefit directly from a more sustainable research software landscape. Last but not least, by using FLOSS research software, libraries could avoid expensive licenses and often insufficiently adapted commercial software.

Infrastructure units, such as supercomputing facilities and university computing centers, benefit from sustainable software as it makes their daily work in terms of software installation and user support easier. Additionally, they can position themselves at the forefront of research by bundling expertise on the creation and maintenance of sustainable research software and installing research software engineering teams.

Industry benefits from sustainable research software, as the process of creating and maintaining research software produces a highly-skilled workforce. Depending on the employed licensing model, sustainable research software can also be adopted by industry partners to reduce cost in corporate research and development. Helping to sustain research software may also enable positive outreach for companies across industry and into society.

Independent (open source) developers can get involved in research software, even if they are not employed by a research institution. This can help them get in contact with other

developers in the field and may potentially lead to collaborations or job opportunities in research based on this extended experience.

How to decide which software to sustain?

Kim's PI is happy because Kim writes a longer section on *hal9k* for the final project report and provides a software management plan alongside it, which ticks off a box in the template that the PI had previously worried about. The PI does not want to let Kim go and instead offers her to be co-PI on a follow-up project to test new methods on the data, and integrate them into *hal9k* as well. They are positive that such a project proposal has a good chance to be funded, as they can show impact of their first project via their university's current research information system (CRIS) and through the number of citations of *hal9k* and the publications for which it was used. While they write the proposal, the faculty dean approaches the two to tell them that based on Kim's work, they will now negotiate about two new RSEs for the central RSE team with the university's provost for research and plan to consider candidates with a background in researchonomics.

When they get the decision letter from the research funding organization, Kim and her co-PI are happy to learn that their new project has won the grant. The reviewers specifically point out the value of extending Kim's Boa library to include the proposed new methods, as well as the significant reuse potential of *hal9k* for the researchonomic community as a direct effect of its well-engineered architecture and modularity. Additionally, they stress that it was really easy to evaluate the software due to the comprehensive test suite, documentation, and example data. In fact, during the first month of the new project, three other researchonomic research projects approach them to ask whether they can contribute to Kim's library and offer to fund six months of RSE work for this. Kim uses this money to also parallelize *hal9k* together with the RSEs and works with her university's computing center to offer it as a standard tool for researchonomic supercomputing.

Requirements and challenges

The sustained funding of all existing software efforts is not only impossible but would risk overly splintering the community and eventually become counterproductive to the efficiency of the research community. Therefore, it is important to agree on a list of transparent criteria that qualify a software product for sustained funding. We recognize that defining research software engineering criteria for software evaluation will also lead to activities aiming at optimizing scores to achieve these criteria. Hence, the criteria have to be designed such that all score-pushing effort truly advances the value of the software. Criteria that can be manipulated without effectively adding value, i.e., wasting resources, should be excluded. The list of criteria presented in this section could be the basis for a structured review process that facilitates an unbiased evaluation of software tools from various fields. Therefore, this list must be general enough to be applied to research software from various research disciplines while also respecting differences between fields (e.g. citation rates between humanities and life sciences). The challenge to do justice to a wide spectrum is e.g. reflected by suggesting criteria comprising different levels⁵⁷. One of the major challenges in the endeavor to define a selection scheme for sustainable funding of research software is to

organize a fair and transparent review process. We believe that it is important that the review process is conducted by experts, or teams of experts, that have a strong background both on software engineering as well as on the domain-specific aspects, the latter because certain criteria often exist on a spectrum that is most likely shaped by the specific demands of the respective research community.

While an assessment based purely on quantitative metrics would allow for seemingly objective comparisons between programs, the definition of valid and robust quantitative metrics that can be evaluated with reasonable effort is a major challenge. On the other hand, a structured qualitative assessment with scores for groups of criteria can provide a middle ground. It is clear that both preparing an application for a review against these criteria from the applicant side as well as the evaluation by the reviewers requires significant effort. We believe that the added value significantly outweighs the investment but appropriate resources need to be factored in. Sustainability of research software should be considered from the beginning for new projects. The criteria listed below, or a subset such as the "good enough" practices proposed by Wilson *et al.*⁴³ and artifact review approaches^{58,59} are valuable throughout the development process (including early phases) for almost all types of research software applications. "Classical" research funding schemes should acknowledge the need to follow best practices during the development of new software and allow factoring in appropriate resources to design and implement for sustainability. In this section, we focus on the question which software to support in dedicated sustainability funding schemes. For such sustained funding, only software in application class 2 or 3 as defined by Schlauch *et al.*⁶⁰, i.e., with significant use beyond personal or institutional purposes, would likely be considered. Excellence as reflected in funded projects, publications, and software adoption, i.e., backing by a community, should be considered during selection. Nevertheless, we believe a good scheme should strike a balance between consolidating the field to few well-established software packages on one side and stimulating innovation and cooperation promoting diversity in terms of more than one monopolistic package on the other side. Last but not least, there is an inherent conflict between the long-term goals of sustainability funding software and the necessary reevaluation to monitor the state of the software over time.

Selection criteria

Several evaluation schemes for research software have been proposed before and led to the formulation of first recommendations^{13,14}. Gomez-Diaz & Recio suggested the CDUR scheme based on Citation, Dissemination (including aspects like license, web site, contact point), Use, and Research (output)⁶¹. Lamprecht *et al.* rephrased the FAIR data principles¹⁰ for research software¹². Hasselbring *et al.* found that the adoption of FAIR principles is different between fields with an emphasis on reuse in computer science as opposed to a reproducibility focus in computational science¹¹. Fehr *et al.* collected a set of best practices for the setup and publication of numerical experiments⁶². Jiménez *et al.* boiled it down to four

best practices⁶³; public source code, community registry, license, and governance. Hsu *et al.*⁶⁴ proposed a framework of seven sustainability influences (outputs modified, code repository used, champion present, workforce stability, support from other organizations, collaboration/partnership, and integration with policy). They found that the various outputs are widely accessible but not necessarily sustained or maintained. Projects with most sustainability influences often became institutionalized and met required needs of the community⁶⁴. In the field of open source software, the CHAOSS (Community Health Analytics Open Source Software) project has developed metrics to evaluate sustainability. One objective of CHAOSS is to automatically generate project health reports based on software that evaluates the metrics, with most of the metrics already covered. The UK Software Sustainability Institute (SSI) suggested both a subjective tutorial-based and a more objective criteria-based software evaluation scheme⁶⁵, the latter being available as an online form. ROpenSci⁶⁶ provides software reviews for R developers, which have been very successful in the community. The review criteria of the Journal of Open Source Software (JOSS) focus on the aspects license, documentation, functionality, and tests. This list of essential items should be fulfilled by all research software that wants to be considered not only for publication but also for sustained funding.

We drew inspiration from all these works and suggest a set of criteria on which to base reviews for sustainable funding. This set comprises mandatory, hard criteria that we think have to be fulfilled across domains (highlighted in italics) and additional desirable, soft criteria that can be implemented to different degrees depending on the use case and domain-specific software development requirements. The soft criteria should be evaluated in a structured way by the reviewers with a specific response for each section rather than one running text. The fact that most of these criteria will be considered in any software management plan (SMP) highlights its importance for sustainable research software.

Usage and impact. Requirements qualifying software for sustained funding are (1) its *use beyond a single research group*, (2) the scientific relevance and validity of the software documented in *at least one peer-reviewed scientific publication*. Ideally a paper also describes the scope, performance, and design of the software. (3) The use of the software in publications is a measure of impact but quantitative assessment brings about additional challenges²⁷. Therefore, other, potentially domain-specific, impact measures, such as influence on policy and practice as well as use in other software and products should be considered as well to evaluate relevance for academia and society. Considerable attendance at training and networking events can be considered as a proof of use as well. (4) A *market analysis* needs to show that the software is important to a user base of relevant size and either unique or one of the main players in a field with several existing solutions. Geographical or political aspects can be considered as well, e.g. to support the maintenance of a European solution. A convergence process of (parts of) a research community towards a specific software stack, i.e., documented transition of several research groups to a

common software, would be a strong indicator of impact. (5) As community uptake and benefits are a central goal of sustained software funding, outreach and *appropriate training material* for new users of the software are essential.

Software transparency and quality. As mandatory criteria of software transparency and quality that have to be fulfilled, we consider (6) the *public availability of the source code* in both a code repository and an archive (for long term availability), developed using (7) *version control* with meaningful commit messages and linked to an issue tracker (ideally maintained, but at least mirrored on a public platform). (8) *Documentation* of the software needs to be publicly available comprising both user documentation (requirements, installation, getting started, user manual, release notes) and developer documentation (with a development guide and API documentation within the code, e.g. using Doxygen)⁶⁷. (9) The *license* under which the software is distributed must be defined. Publicly funded software should be published under a Free/Libre Open Source Software (FLOSS) license by default, although exceptions to this might apply (e.g. excluding commercial use). (10) *Dependencies* on libraries and technologies must be defined.

We acknowledge that some additional criteria have to be evaluated under consideration of the research domain. These comprise (11) the availability of examples (comprising input data and reference results), (12) mechanisms for extensibility (software modularity) as one aspect of software architecture⁶⁸ and (13) interoperability (APIs / common and open data formats for input and output), (14) a test suite (including at least some of the following: unit tests, regression tests, integration tests, end-to-end tests, performance tests; ideally run in an automated fashion in a continuous integration environment), (15) tagged releases (considering their frequency, and availability for end users in terms of binary packages for major operating systems, or availability via package managers or containers), (16) no large-scale re-implementations for functionality for which good solutions already exist. Many of these aspects require appropriate infrastructure (see page 12).

Maturity. The research software applying for sustained funding must have already reached a certain level of maturity (typically class 2 or 3 as defined by Schlauch *et al.*⁶⁰). A mandatory requirement is (17) a comprehensive and up-to-date *software management plan*⁶⁹. The software should (18) be maintainable with an appropriate amount of resources as detailed in a sustainability section of the software management plan. The software has (19) a well maintained website with a clearly defined *point of contact* and a communication channel to inform users about news regarding the software such as new releases. Besides an active user community, sustainable software requires (20) a group of developers (i.e., definitely *more than 1 developer*) documented, e.g. by contributions to the code base or participation in documented, public discussions or issue tracking. Another criterion is (21) whether potential contributors are invited to participate in a clearly defined process (e.g., a CONTRIBUTING document). The group of developers should have defined a governance model for their project and easy ways for users to provide input regarding their needs.

Recommendations

Given the diversity in the software technology landscape, and the domain-specific software development cultures⁷⁰, some of the above-mentioned criteria have to be evaluated against domain-specific requirements. Therefore, we highly recommend to base the selection process on a combination of (1) a software quality-based review and (2) a domain-specific scientific review. In particular, the former should be ideally performed by a central institution (e.g. at funding bodies or other independent agencies such as a software sustainability institute). Only criteria for which improvement truly advances the value of the software should be considered in evaluation schemes, i.e. no criteria that can be gamed. After rejecting software not fulfilling the mandatory criteria in a first stage of the review process, the second stage of the selection process should be realized as a transparent procedure ideally allowing the reviewers to interact with the PIs of the software (e.g. remote meetings, forum-like discussions) and put the software quality and development efforts into the domain-specific context. The outcome of this second stage should be a structured review assessing each criterion explicitly and a rating for each of the dimensions *Usage and impact*, *Software quality*, and *Maturity*. For sustained software funding, it is important to audit the performance, relevance, impact, progress, and level of sustainability of funded software after reasonable time frames. Such a reevaluation should revisit the criteria under consideration of evolving software technology and scientific standards, without requiring a completely new proposal being submitted. We envision funding periods of 5 years to provide sufficient security for funded software projects, while allowing for adaptation of the portfolio of funded software to novel research directions and community needs. Failure to meet the reevaluation criteria should lead to the decision to phase-out sustainable funding. The phase-out process may come with a 1-year funding program based on a consolidation plan with clear goals regarding the archiving and preservation of the software, documentation, and all existing resources.

Who sustains research software?

Kim wants to broaden her research portfolio within researchonomics and applies for postdoctoral positions at other institutions. Her library *hal9k* is growing in popularity within researchonomics, and she wants to continue working on it. As her university has adopted an open science policy, *hal9k* is free software under a Free/Libre Open Source Software (FLOSS) license, and Kim is free to continue her work on the library even after moving away from UoFA. Due to her involvement in the creation of *hal9k* as well as her previous success in attracting funding, Kim has the choice between multiple, attractive positions and decides to move to the researchonomics group at Eden University (EdU). She has already extended *hal9k* in multiple directions in the past and plans to continue this work at EdU. Her group leader at EdU would like to continue funding her but due to a law called the Fixed-term Research Contract Bill, EdU is not allowed to extend her contract, and neither third-party funding for her own position nor a permanent position are available. After having developed a now widely-used research tool, several publications in software and paper form, as well as having attracted funding, Kim finds herself looking for a job again.

Research relies on software and software relies on the people developing and maintaining it. Sustainable research requires sustainable software, and this in turn requires continuity for those who develop and maintain it.

Requirements

Possibly the most important demand is the need for an increase in *recognition and awareness* of research software as a first class citizen in research^{14,71,72}. For sustainability of research software, long-term commitments of the respective software leads are crucial, but very few *professional RSE profiles* currently exist. In consequence, it is essential to create career paths for RSEs that are attractive and include permanency perspectives. While creating permanent positions in the German academic system below the faculty level is an actively discussed topic overall⁷³, we specifically focus on the needs originating from the development and maintenance of research software here.

As already mentioned, research software development not only requires domain expertise, but also software development *education, skills, and competence*. Currently, most of the domain researchers developing and maintaining domain-specific software technology have not received professional training on software development^{3,41}. To enhance the productivity and sustainability of computer-based research, it is essential to integrate software development training into the education of domain researchers.

Currently, a significant portion of the existing research software is developed by individuals or in small groups, primarily to serve their own requirements. This situation is unsatisfying in terms of collaboration and inefficient in terms of several groups spending resources on generating similar or even the same functionality. To enable and promote synergies, it is important to allocate resources for research software development and to build *communities*, as described in 74.

Challenges

We are currently facing a *lack of awareness* for the importance of research software as discussed above. Moreover, there is *little recognition* for the efforts put into software development and maintenance. In consequence, software development in academic settings is mostly considered as a means to an end and sustainability is often not considered in project planning and grant proposals and contributes little to progressing research careers⁷⁵. The main challenge here is the continued use of metrics that primarily leverage traditionally published articles and article citation numbers.

In academia, developers of research software are typically domain researchers, and in particular if new areas are explored, the software development process itself has research character. Obviously, developing research software requires not only domain knowledge but also software development skills, and the researchers leading the software development process are often domain experts with substantial software development experience, making them extremely valuable members of the research

community. However, the current academic system in Germany does not provide a defined *RSE role*. Fixed-term positions are, at least currently within the German academic system, often effectively the end of a Research Software Engineer's career path, sometimes even a dead end. The challenge here is the lack of available permanent positions within the non-professorial academic faculty ("Mittelbau") in Germany, compounded by a lack of access to these few permanent positions for RSEs. This in turn is due to the already mentioned lack of recognition for efforts concerning research software for faculty appointments within domain sciences.

In order to develop sustainable software, researchers need to have the *skills and expertise* to build software that is easy to maintain and extend⁷⁶. However, most of the researchers are self-taught developers^{3,41}. Ideally, these skills have to be built into the domain science curricula, which could generally be done in two different ways (or a combination of them). One obvious solution attempt are additional courses that focus on these topics. The main challenge here is to decide which other topic(s) to possibly drop due to the limited volume of any given curriculum. A different approach is to incorporate software-related topics into existing domain science courses. While this would provide the benefit of show-casing the usage of specific software skills directly within the domain science, the challenge here is the amount of work necessary to change existing lecture material, let alone the need of the lecturers to acquire those skills themselves in the first place.

As long as the necessary software skills within domain sciences are not yet wide-spread, building a network from those that have acquired relevant skills is difficult. *Community* efforts, that concentrate on questions regarding research software, can help to fill this gap. Examples of such efforts include the Software Carpentries, national and international RSE societies (e.g., within Germany *deRSE* e.V.). However, since research software is such an interdisciplinary topic, it is hard to get recognition and find funding within any specific discipline. As a result, existing communities often have to rely heavily on volunteers. This is challenging because despite benefits to domain science, volunteers hardly receive recognition for their work "back home", i.e., within their domain, underlining again the importance of our first demand.

Recommendations

Increasing *recognition and awareness* is a challenge that calls for both immediate action and perseverance. Nevertheless, some measures will likely show positive effects comparatively soon.

Similarly to plans for research data management, funding agencies should request that applicants include considerations about how software developed in a project can be sustained beyond the end of the funded project. A follow up on these plans during and after the project lifetime, i.e., a dedicated software management plan, is crucial.

Another recommendation is aimed at decision makers concerning recruitment for academic positions: broaden the

definition of research impact beyond traditional scientific publications to also include other impactful results. Not all researchers that think of themselves as RSEs pursue a faculty position as their main career goal. However, permanent academic non-faculty positions are rare within the German academic system, also due to the lack of a defined *RSE role*. We recommend research institutions to leverage the benefit of dedicated RSEs by establishing attractive long-term career options in the academic environment. The long-term solution in order to gain sufficient software development *skills* should be education that is included early in the career path, ideally already at the Bachelor level. For the time being however, efforts involving workshops and seminars that provide easy access to hands-on training on software-related questions should be promoted and supported as much as possible.

It is important to provide an environment where *communities* can form and flourish by allocating resources for research software development and for building communities around it^{63,74,77}. The identification with a community of like-minded people and personal action⁷⁸ can lead to a permanent establishment of sustainable research software as a valuable research output. Thus, research institutions as well as funding agencies should not only be open-minded regarding existing volunteer organizations, but should actively promote the creation of such groups.

How can research software be sustainably funded?

hal9k has grown into a widely used software in researchonomics, and Kim is proactively asked to apply for - and is subsequently awarded - a permanent RSE position at the institute for researchonomics at UoFA, based on her work on the library. She works closely with the central RSE team, but mostly due to bureaucracy and the high demand for her library, Kim does not have enough time to maintain and further develop *hal9k* alone anymore. Together with the dean she develops a course for the researchonomics curriculum which teaches data processing with *hal9k*. As a lesson from her own career, she starts the course with sessions on the Hash shell, version control with Tig, Boa, and two whole sessions on basics of sustainable software development. This is very fruitful, and due to the implementation of a new research software funding scheme at UoFA, Kim is able to hire one of the course students, who has shown great RSE skills, straight into a long-term position at her institute, where they focus on the maintenance and development of *hal9k*, work with the computing center to support *hal9k*-based supercomputing on a new, dedicated FGPA cluster, develop training materials for external users, and organize the yearly *hal9k* users and developers conference. Kim gets to travel the world to visit researchonomics groups who are using *hal9k*.

Requirements

Sustainable funding for research software boils down to funding the four main pillars enabling sustainable software development: (1) Personnel with expertise in research software development; (2) Infrastructure for developing, testing, validating, and benchmarking research software; (3) Training in software design and sustainable software development; and (4) Community management and events for creating synergies between research groups and software efforts.

Challenges

Short-term engagement of (early career) researchers raises the question of how to maintain a constant level of expertise within a developer team and prevent knowledge drain concerning domain knowledge and software engineering skills. Conversely, the permanent engagement of qualified personnel requires to offer career perspectives, especially due to the fact that academia competes with industry for the same people. A challenge specific to Germany is posed by the shortage of permanent positions and by the restrictions for temporary positions due to the German *Wissenschaftszeitvertragsgesetz*⁷⁹.

Sustainable software development requires hardware technology to develop, test, validate, and benchmark features in a continuous integration cycle. The challenge in this context is the persistent evolution of the hardware landscape. Hence, for creating an environment promoting sustainable software development, it is important to provide access to a wide hardware portfolio and to support a development cycle based on continuous integration.

Expertise in sustainable research software development is a scarce resource, and training is heavily needed as one way of building up more expertise. However, while integrating interdisciplinary software engineering courses into the education curriculum can build up basic skills, some expertise is domain-specific and requires interinstitutional training activities. Furthermore, there exist no financial incentives for creating software-specific documentation and tutorials nor to provide other forms of support.

While the creation of research software communities is one major asset in sustaining research software technology, promoting this process requires the installation of new funding instruments. Traditionally, research grants are limited to rather short time frames and support personnel, material, hardware, and to a limited degree also travel and research visits. Creating a research software community however requires funding for community and training events as well as “virtual hardware” such as webspace, versioning systems, task-managing systems, and compute cycles. These demands can hardly be met without third-party funding^{45,80–82}.

Recommendation: creation of adequate funding schemes

Funding is a crucial factor for sustaining research software. Currently available sources and instruments are not adequately shaped for the challenges and solutions outlined above. We recommend actions on the individual, organizational, and national level.

Existing project-focused funding instruments on the local, national, and international level need to be complemented with funding instruments specifically designed for research software development and sustained research software maintenance to make research software a first class citizen in the research landscape. For example, software projects enhancing research and fulfilling the sustainability criteria detailed in section *How*

to decide which software to sustain? may be entitled for sustained funding as long as they live up to the standards and remain a central component of the research landscape.

Computing centers and supercomputing facilities for research need to receive earmarked resources for the support of sustainable software development. This funding is necessary to provide continuous integration services, a hardware portfolio for development, testing and benchmarking software, as well as personnel for training domain researchers in software design and the proper usage of the services.

The creation and maintenance of training materials for general research software engineering education and the software-specific documentation and tutorial creation needs to be reflected in funding opportunities. This can either happen by dedicating modules of research or software grants to providing support and the generation of training material, or by opening funding schemes focusing on interdisciplinary software development education. The latter may include research that looks at research software development as a process to analyze which measures, interactions, and team compositions make research software successful. Additionally, funding instruments fostering the formation of research software communities have to be established.

Which infrastructure is needed to sustain research software?

As the *hal9k* community grows, so does the need for infrastructure. Kim and her team collaborate with the National RSE Consortium to set up *hal9k* on the Consortium's distributed TigHub instance and organize world-wide access to it via the NRSEC-AAI federation. Going forward, the Consortium's Research Software Hub - a registry and [Software Heritage Archive-based](#) long-term repository for research software on a national level - ingests *hal9k* releases with complete metadata: citation information, the *hal9k* provenance graph and computational environment information, [ORCID](#) iDs, etc. and provides its own DOIs for versions under a concept (umbrella) DOI. The community reviews all code and documentation changes that are contributed to *hal9k* via the central TigHub instance. The Hub's CI system Alfred builds, tests, and pushes new releases automatically to the registered supercomputing clusters. Community efforts become better and more streamlined by the day, as research software development training is now offered as part of most curricula, and skilled RSEs are now much easier to find and hire by research institutions.

Project management tools

Research software is developed by individual researchers, in small teams within a single institution, or in larger teams distributed across multiple institutions. In particular if software development is distributed across institutions, there exists an urgent need for frameworks and tools enabling collaborative code development, software feature planning, and software management. As research software development typically includes bleeding-edge research and in some cases development that the researchers do not want to disclose for a certain time to preserve intellectual property, distributed research software development

also needs a global Authentication and Authorization Infrastructure (AAI). We recommend the development and/or deployment of tools for distributed software development and software management as central research infrastructure. An important aspect in this context is the cataloging of research software to reduce the duplication of development efforts. This can efficiently be realized by promoting the registration of all research software with a unique identifier and developing a tool that allows to explore the research software landscape. Research software contributors should have an **ORCID** iD to be uniquely identifiable and referable. While some funding for such tools and software repositories is emerging (e.g. the bio.tools catalogue of bioinformatics tools funded as part of the European ELIXIR project⁸³), a standardized extension of such efforts to the RSE community as a whole is necessary. However, as the experiences from ELIXIR demonstrate, this is a non-trivial effort that requires significant dedicated and long-term funding.

Developer training, motivation, and knowledge exchange

As elaborated, training in sustainable software development is key to achieve sustainability in research software. At the same time, it is not clear how such training should be facilitated and institutionalized. Furthermore, for deriving software quality standards, evaluating the quality of software, and providing a code review service, central resources are necessary that individuals and groups in the research software landscape can draw from.

We consider Software Carpentry and similar efforts like the creation of the Data Science Academy **HIDA** in the Helmholtz Association of German Research Centers helpful solutions to exchange and distribute knowledge. Local chapters of RSE groups and (inter-)national conferences will further foster networking and community building. We strongly recommend the creation of a national Software Sustainability Institute (involving funded positions to establish web platforms and training material) similar to the **UK Software Sustainability Institute (SSI)**, which serves as a national contact for all aspects related to research software. The UK SSI also publishes **best practice guidelines** for research software engineering.

Research software discovery and publication

Proper software publication and possibilities for the community to find existing software solutions for a given problem are a prerequisite to optimally exploit synergies and avoid redundant development. However, we observe that today, many funding proposals lack a thorough state-of-the-art report of software that could possibly be reused. This is most often caused by insufficient information retrieval strategies, lack of knowledge about relevant repositories, and an abundance of locations where software is collaboratively developed and stored⁸⁴. Discovery requires publication in a globally accessible location with appropriate metadata, e.g. Citation File Format (CFF)⁸⁵ and **CodeMeta**. Comprehensive metadata (e.g. contributors, contact, keywords, linked publications, etc.) and publishing platforms have to enable persistent citing, which in turn benefits research evaluation. Selection and curation of software (probably by a data/software librarian) for publication and discovery are certainly challenging.

We consider GitLab or GitHub as collaborative working environments and repositories like Zenodo appropriate publication platforms, because the latter mint DOIs, allow versioning and are publicly funded for long-term access. GitHub, Figshare, and Mendeley Data are examples of commercial enterprises with business cases in the background, which leverage research results. Besides the aforementioned metadata standards, it is advisable to document source code, e.g. using Markdown (with Doxygen tooling). Metadata and citations play a role in beneficial tools like PIDgraph, DataCite.org, CrossRef, which utilize Persistent Identifiers (PIDs) like DOIs. Another solution to discovery are (mostly) disciplinary software indices like **swMATH** or the **Astronomy Source Code Library** as well as language focused systems like **CRAN** for R. Most of them started as national endeavors and became platforms of global importance. For Germany, we assume that the Nationale Forschungsdateninfrastruktur (NFDI) will put effort into creating or supporting discovery platforms at a central point that ease information retrieval. At the same time, all stakeholders should be aware of and counteract potential institutional “fear” of losing “their” data, software, and intellectual property.

Especially in interdisciplinary environments, it would be helpful to have access to a meta software repository index, similar to what re3data⁸⁶ does for research data repositories. We recommend the creation of such a meta index covering important (disciplinary) software indexes in order to ease discovery of relevant software locations. Evaluation of discovered software is an unsolved problem. Here, anonymous telemetry of usage may provide information for the selection of relevant software. Publishing software, their dependencies, and environment in containers may also ease evaluation and further reuse. These suggestions require significant investment in longterm infrastructure. When publishing research software it is recommended to make use of integration schemes like GitHub with Zenodo or local GitLab instances with publication platforms. Such indices and publication outlets may benefit national federated research indexing & archiving systems, similar to the hierarchy of library catalogs⁸⁷.

Archiving

Software preservation aims to extend the lifetime of software that is no longer actively maintained. There are different approaches, which vary in the effort required and the likelihood of success. Software archiving is one important aspect of software preservation: the process of storing a copy of a software package so that it may be referred to in the future. The publication of a certain software version for reference in research articles requires simple ways to archive research software on a long-term basis. Furthermore, its integration with collaborative software development environments such as GitLab or GitHub and with publication repositories is needed to facilitate archiving of referenced software versions based on sustainable frameworks (e.g. **Invenio** for GitHub to Zenodo integration).

A challenge for software archiving is the need to (ideally) preserve the runtime environment and all dependencies of the software. This could improve reproducibility, especially when running the software in its original state. If research data are

needed to reproduce results, they should also be archived with the software or the publication. Specialized and unique hardware like high performance computing resources can be part of the runtime environment, which may not be accessible in the future. To overcome this, an emulation of hardware may be a (challenging) solution. Emulation involves the encapsulation and distribution of the complete hardware and software stacks, including the operating system and driver interdependencies. This can result in intellectual property issues when offered as a service.

There are both local and global approaches to software conservation. One solution to keep the software in an executable state by preserving its context and runtime environment is to use containers such as Docker. However, to archive the Docker containers, additional metadata should be added and stored with the software in an archive container format that allows exchange between repositories and exit strategies, such as the BagIt container format⁸⁸. Application or platform conservation is also achieved by conservational efforts where unmaintainable (virtual) machines are sandboxed to keep the platform in a secure but running state. Other notable efforts in this direction include for example Singularity and Guix HPC. Another threat is losing project repositories on global platforms like Github or BitBucket. Here, global platforms like [Software Heritage](#) harvest those repositories and prevent loss by long-term archiving.

Legal aspects

More and more industrial partners enter the *hal9k* community, and they bring their lawyers. Together with UofA's research software task force, the RSE team, the researchonomy institute, the corporate lawyers, and community representatives, Kim decides to create a foundation to govern *hal9k* and its environment: the Fullest Possible Use Foundation for Open Researchonomy, funded by the Ministry of Research and Education and a consortium of corporate partners. As a first step, they re-license *hal9k* under the OSI-approved [MIT license](#).

A common situation in research software creation is that the developer has no knowledge or awareness of legal aspects and therefore did not consider them early enough. As seen in Kim's example, re-licensing later in the project can be not only legally, but also organizationally very tricky, in particular for projects which developed over many years and involved many contributors from different organizations. Thus, we think the main legal demands for research software development are raising awareness and empowering all levels of responsible persons in academia (from researchers and RSEs over PIs to research performing organizations and research funding organizations) in legal aspects. This will hopefully lead to a general legal certainty before, during, and after the research software development process and thus enable better options for collaborations between universities, non-commercial research institutions, and other national or international partners. Legal aspects always have to be considered regarding the relevant jurisdiction. Though similar issues arise in all jurisdictions, the following will focus on the European and specifically German legal framework.

Challenges and clarifications

Clarification of rights. Software development is a creative activity. The main relevant law governing legal aspects is therefore the copyright law. It regulates the rights and obligations of the parties involved. Chapter 8 of the German Act on Copyright and Related Rights (UrhG) contains specific provisions applicable to computer programs and is based on the EU computer programs directive. Copyright law protecting the creator of software in similar ways exist in nearly all legal systems. It is important for the identification of rights that software, in the sense of (German) law, includes not only the source code but also the design materials⁸⁹. The challenge in the use, distribution, and commercialization of software is to determine the chain of rights and to identify all right holders. The owner of the copyright is not necessarily the owner of the right of use. For Germany, the Copyright Act regulates the rights for employment relationships⁹⁰. In such cases, the right of use is automatically transferred to the employer. This means that in most cases of employed software developers and research staff, the institution holds the rights of use for the software work. This is not automatically the case for students, freelancers, and individual external cooperation partners. Employment and service contracts with contributors could contain regulations regarding the transfer of rights of use. For researchers who conduct free research not subject to directives, in Germany the constitution guarantees freedom of research so that the rights of use for their work remains initially with the natural person. In addition to the rights of the people directly involved, other rights of third parties may also be relevant. Existing source code (e.g., other Free/Libre Open Source Software (FLOSS)), external libraries, and contributions from institutional cooperation partners are published and provided under certain licenses and their conditions must be observed (which, due to incompatibilities even among FLOSS licenses, may well mean that individually reusable pieces of software cannot be reused together or in a new context). The nature of research careers often brings additional complications to the chain of rights. It happens that researchers take their software with them when they change institutions and develop it further during their career. Here, the former employer may be entitled to some rights of use. In third-party funded projects, in particular with industry but also with public funding, rules regarding rights of use are often defined. Last but not least, the software can also be affected by other (intellectual) property rights such as patents or trademarks. Software itself is usually not patentable but it may implement a technical invention covered by patents. When using or distributing such software, an additional matching patent license may be necessary. Licenses exist (for example: GNU GPL v3) which automatically grant related patent licenses while using the software license. That should be considered when exploitation of the patent is planned.

Liability. Issues of warranty and liability for faulty software must be taken into account. We consider the possibilities of contractual limitation of liability in licenses. Full exclusions of liability are generally invalid in the German law. Limitations of liability usually depend on the form of distribution: The limitation options are larger if the rights of use are granted free of charge, e.g. provision "as is" as defined, e.g. in the BSD 3-clause license.

Ideas for solutions

In order to meet the legal challenges mentioned, it is absolutely necessary for the software developer (team) to document the rights chain comprehensively during the software development (one possible solution is presented in the accompanying report⁹¹). Contributions of individual persons must be traceable and their (labor law) status must be named. At best, contracts with rules on the transfer of rights of use should be concluded before work begins. Declarations of assignment of rights can be made for existing works. License conditions for external contributions must be evaluated with regard to further rights of use and possible sub-licensing. Contracts and funding conditions must be conscientiously documented and analyzed with regard to rules on rights of use. In case that different parts of the software are based on different conditions and rights of third parties, individual modules of the new software could be published under different licenses and merged accordingly.

A national research software sustainability institute could be established. This institute supports local research software task forces and thereby respective researchers and research teams in the licensing of research software and related legal issues. For this purpose, a legal help desk will be set up, to which all members of their respective research performing organization can apply. Such a legal help desk should be seen as an infrastructural investment to avoid any uncertainty about re-use of existing research software and to support research-friendly licensing. If researchers want to publish the research software under a Free/Libre Open Source Software (FLOSS) license, the organization could bundle the necessary rights beforehand. This is particularly useful when teams of researchers, often international, write software. In addition, the sustainability institute may serve as a one-stop-shop for the licensing of research software.

Recommendations

We see it as an essential part of the sustainability of research to enable the free distribution of research software. There are a variety of open source software licensing models (ranging from permissive to copyleft; for further information, see [tldrlegal](#), the [ifrOSS Lizenz-Center](#), or Morin *et al.*, 2012⁴⁸). The use of an FSF- or OSI-approved FLOSS license for example would enable a truly free model and also reduce legal issues. We recommend that research funding organizations such as the DFG discuss if they expect publishing all funded software under these licenses, following the paradigm of “public money, public code”. If licenses such as Apache or MIT are applied, the research institutions may later still commercialize the software if appropriate. Such open source licensing is also beneficial for start-ups that intend to provide professional services for the software.

Also for legal aspects, we believe it is important that all (German) research performing organizations install a research software task force, especially in light of the new [DFG Code of Conduct](#). Besides organization and bundling of technical and infrastructural support for local RSEs and researchers (see previous sections), this group should organize a local legal help desk, organize educational offers e.g. for the legal topics

presented, and (if not implemented yet) develop the software policy of the research performing organization. As an example, with the help of on-boarding processes performed by the research software task force, RSEs should be able to keep the clearance of rights as simple as possible right from the start. This helps to avoid that - out of uncertainty and fear to make a legal mistake - some research groups end up not choosing any license at all, which may hinder reuse of the software. We suggest that the local task forces build a network with the other research performing organizations for exchange of ideas but also for generating a bottom-up strategy to organize RSE standards for Germany and beyond and possibly be the origin of the aforementioned software sustainability institute.

Conclusions

We find that the research software ecosystem is notoriously lacking resources despite its strategic importance. If funding and support does not improve, the success story of science based on academic research software may be at stake. We recommend the installation of infrastructure that enables sustainable software development including platforms for collaboration, continuous integration, testing, discovery, and long-term preservation. We suggest the establishment of a nationwide institution similar to the Software Sustainability Institute (SSI) to provide project consulting and code review services as well as sustainable software development training. We think that sustainable software development should become an integral component of the universities’ teaching curriculum. We encourage the research funding bodies to reflect the licensing models for academic software development, and to decide whether the “public money, public code” paradigm justifies the requirement that all publicly funded software has to be publicly available under a Free/Libre Open Source Software (FLOSS) license. Ultimately, we strongly advise the implementation of funding schemes for sustainably supporting the development and maintenance of research software based on clear and transparent criteria, for creating incentives to produce high quality community software, and for enabling career paths as research software engineer (RSE).

Glossary

domain researchers The people doing the research to advance knowledge in a field.

general public Lay people that do not necessarily have specific insight regarding a research domain.

geopolitical units Governed public units, ranging from cities and councils, over federal states and countries, up to political unions such as the EU. In the context of this paper, the discussion usually focuses on the larger units (countries and political unions).

independent (open source) developers Project-external software developers who are not employed by the institution(s) carrying out the project.

industry Companies conducting research or profit from available academic research software which they can directly or indirectly apply to their field.

infrastructure units Computing centers of research bodies such as universities and other research centers, as well as high-performance computing facilities.

libraries (also registries, indices) Infrastructure units of research bodies such as universities, or independent organizations, which gather research outputs and their structured metadata, and provide indices, search, etc.

research funding organizations Public research funding bodies but potentially also companies, foundations, associations, etc.

research leaders Heads of research groups, such as professors and other people with staff responsibility.

research performing organizations Research groups, departments, faculties, research institutions (universities, national labs, cross-institutional research groups, etc.), umbrella organizations, such as Helmholtz-Gemeinschaft Deutscher Forschungszentren, Max-Planck-Gesellschaft zur Förderung der Wissenschaften, Leibniz-Gemeinschaft, etc.

research software engineers (RSEs) People creating and maintaining research software; this group ranges from research-focused software developers, to software engineers with a focus on research; other definitions include other roles, such as research software managers.

Data availability

Underlying data

No underlying data are associated with this article.

Extended data

Decision trees and documentation templates for the legal topics are available on Zenodo⁰¹: doi: [10.5281/zenodo.4327147](https://doi.org/10.5281/zenodo.4327147)

Author information

We are a group of software-providing researchers, RSEs, and infrastructural as well as legal supporters. Initially, a group of representatives of funded projects of **funded projects of the first DFG sustainability call** met during the **first German RSE conference (deRSE19)** in June 2019 in a grass-roots workshop on sustainable research software addressing the software-based research community. During this workshop, we realized that a lot of valuable experience and good ideas are present in the group, and we decided to start working on this paper together with other interested practitioners. We followed the generous invitation of the DFG for the above-mentioned two-day meeting at the Robert Koch Institute in Berlin in November 2019 to sharpen the focus of this paper.

References

1. The Event Horizon Telescope Collaboration, Akiyama K, Alberdi A, *et al.*: **First M87 Event Horizon Telescope Results. IV. Imaging the Central Supermassive Black Hole.** *Astrophys J.* 2019; **875**(1): L4.
[Publisher Full Text](#)
2. Nowogrodzki A: **How to support open-source software and stay sane.** *Nature.* 2019; **571**(7763): 133–134.
[PubMed Abstract](#) | [Publisher Full Text](#)
3. Philippe O, Hammitzsch M, Janosch S, *et al.*: **softwaresaved/international-survey: Public release for 2018 results.** 2019.
[Publisher Full Text](#)
4. Hirsch JE: **An index to quantify an individual's scientific research output.** *Proc Natl Acad Sci U S A.* 2005; **102**(46): 16569–16572.
[PubMed Abstract](#) | [Publisher Full Text](#) | [Free Full Text](#)
5. Bangerth W, Heister T: **Quo Vadis, Scientific Software?** *SIAM News.* 2014; **47**(1): 8.
[Reference Source](#)
6. Prins P, de Ligt J, Tarasov A, *et al.*: **Toward effective software solutions for big biology.** *Nat Biotechnol.* 2015; **33**(7): 686–687.
[PubMed Abstract](#) | [Publisher Full Text](#)
7. Richardson C, Croucher M: **Research Software Engineer: A New Career Track?** 2018.
[Reference Source](#)
8. Cohen J, Katz DS, Barker M, *et al.*: **The Four Pillars of Research Software Engineering.** *IEEE Software.* 2020; **38**(1): 97–105.
[Publisher Full Text](#)
9. Brett A, Croucher M, Haines R, *et al.*: **Research Software Engineers: State of the Nation Report 2017.** 2017.
[Publisher Full Text](#)
10. Wilkinson MD, Dumontier M, Aalbersberg IJ, *et al.*: **The FAIR Guiding Principles for scientific data management and stewardship.** *Sci Data.* 2016; **3**(1): 160018.
[PubMed Abstract](#) | [Publisher Full Text](#) | [Free Full Text](#)
11. Hasselbring W, Carr L, Hettrick S, *et al.*: **FAIR and Open Computer Science Research Software.** arXiv: 1908.05986. 2019.
[Reference Source](#)
12. Lamprecht AL, Garcia L, Kuzak M, *et al.*: **Towards FAIR principles for research software.** *Data Sci.* 2019; **3**(1): 37–59.
[Publisher Full Text](#)
13. Katerbow M, Feulner G: **Recommendations on the development, use and provision of Research Software.** 2018.
[Publisher Full Text](#)
14. Scheliga K, Pampel H, Konrad U, *et al.*: **Dealing with research software: Recommendations for best practices.** Helmholtz Open Science Coordination Office. 2019.
[Publisher Full Text](#)
15. Hatton L: **The Chimera of Software Quality.** *Computer.* 2007; **40**(8): 104–103.
[Publisher Full Text](#)
16. Chang G, Roth CB, Reyes CL, *et al.*: **Retraction.** *Science.* 2006; **314**(5807): 1875–1875.
[PubMed Abstract](#) | [Publisher Full Text](#)
17. Matthews BW: **Five retracted structure reports: inverted or incorrect?** *Protein Sci.* 2007; **16**(6): 1013–1016.
[PubMed Abstract](#) | [Publisher Full Text](#) | [Free Full Text](#)
18. Kanewala U, Bieman JM: **Techniques for testing scientific programs without an oracle.** In: *2013 5th International Workshop on Software Engineering for Computational Science and Engineering (SE-CSE).* 2013; 48–57.
[Reference Source](#)
19. Vogel T, Druskat S, Scheidgen M, *et al.*: **Challenges for Verifying and Validating Scientific Software in Computational Materials Science.** In: *International Workshop on Software Engineering for Science.* 2019; 25–32.
[Publisher Full Text](#)
20. Smith AM, Katz DS, Niemeyer KE, *et al.*: **Software Citation Principles.** *PeerJ Comput Sci.* 2016; **2**: e86.
[Publisher Full Text](#)
21. Merali Z: **Computational science: ...Error.** *Nature.* 2010; **467**(7317): 775–777.
[PubMed Abstract](#) | [Publisher Full Text](#)

22. Barnes N: **Publish your computer code: it is good enough.** *Nature*. 2010; **467**(7317): 753.
[PubMed Abstract](#) | [Publisher Full Text](#)
23. Tse H: **Computer code: more credit needed.** *Nature*. 2010; **468**(7320): 37.
[PubMed Abstract](#) | [Publisher Full Text](#)
24. Hafer L, Kirkpatrick AE: **Assessing Open Source Software As a Scholarly Contribution.** *Commun ACM*. 2009; **52**(12): 126–129.
[Publisher Full Text](#)
25. Howison J, Bullard J: **Software in the Scientific Literature: Problems with Seeing, Finding, and Using Software Mentioned in the Biology Literature.** *J Assoc Inf Sci Technol*. 2016; **67**(9): 137–2155.
[Publisher Full Text](#)
26. Li K, Yan E, Feng Y: **How Is R Cited in Research Outputs? Structure, Impacts, and Citation Standard.** *J Informetr*. 2017; **11**(4): 989–1002.
[Publisher Full Text](#)
27. Li K, Chen PY, Yan E: **Challenges of measuring software impact through citations: An examination of the lme4 R package.** *J Informetr*. 2019; **13**(1): 449–461.
[Reference Source](#)
28. Park H, Wolfram D: **Research software citation in the Data Citation Index: Current practices and implications for research software sharing and reuse.** *J Informetr*. 2019; **13**(2): 574–582.
[Publisher Full Text](#)
29. Pan X, Yan E, Cui M, et al.: **How Important Is Software to Library and Information Science Research? A Content Analysis of Full-Text Publications.** *J Informetr*. 2019; **13**(1): 397–406.
[Publisher Full Text](#)
30. Doerr A, Rusk N, Vogt N, et al.: **Giving Software Its Due.** *Nat Methods*. 2019; **16**(3): 207–207.
[PubMed Abstract](#) | [Publisher Full Text](#)
31. Druskat S: **Software and Dependencies in Research Citation Graphs.** *Comput Sci Eng*. 2020; **22**(2): 8–21.
[Publisher Full Text](#) | [Free Full Text](#)
32. Katz DS, Bouquin D, Hong NP, et al.: **Software Citation Implementation Challenges.** arXiv: 1905.08674. 2019.
[Reference Source](#)
33. Resnik DB, Morales M, Landrum R, et al.: **Effect of impact factor and discipline on journal data sharing policies.** *Account Res*. 2019; **26**(3): 139–156.
[PubMed Abstract](#) | [Publisher Full Text](#) | [Free Full Text](#)
34. Vandewalle P: **Code Sharing Is Associated with Research Impact in Image Processing.** *Comput Sci Eng*. 2012; **14**(4): 42–47.
[Publisher Full Text](#)
35. Venters CC, Jay C, Lau LMS, et al.: **Software Sustainability: The Modern Tower of Babel.** In: *Proceedings of the Third International Workshop on Requirements Engineering for Sustainable Systems Co-Located with 22nd International Conference on Requirements Engineering (RE 2014)*, Karlskrona, Sweden: CEUR-WS. 2014; **1216**: 7–12.
[Reference Source](#)
36. Goble C: **Better Software, Better Research.** *IEEE Internet Comput*. 2014; **18**(5): 4–8.
[Publisher Full Text](#)
37. Druskat S: **A Proposal for the Measurement and Documentation of Research Software Sustainability in Interactive Metadata Repositories.** In: *Proceedings of the Fourth Workshop on Sustainable Software for Science: Practice and Experiences (WSSSPE4)*, Manchester, UK: CEUR-WS. 2016; **1686**.
[Reference Source](#)
38. Katz DS: **Fundamentals of Software Sustainability.** 2018.
[Reference Source](#)
39. Akhmerov A, Cruz M, Drost N, et al.: **Raising the Profile of Research Software: Recommendations for Funding Agencies and Research Institutions.** NWO (The Netherlands Organisation for Scientific Research). 2019.
[Reference Source](#)
40. Casties R, Czmiel A, Damerow J, et al.: **DH Research Software Engineers - For We Are Many.** 2019.
[Reference Source](#)
41. Wilson G, Aruliah DA, Brown CT, et al.: **Best practices for scientific computing.** *PLoS Biol*. 2014; **12**(1): e1001745.
[PubMed Abstract](#) | [Publisher Full Text](#) | [Free Full Text](#)
42. Stodden V, Miguez S: **Best Practices for Computational Science: Software Infrastructure and Environments for Reproducible and Extensible Research.** *J Open Res Softw*. 2014; **2**(1): e21.
[Publisher Full Text](#)
43. Wilson G, Bryan J, Cranston K, et al.: **Good enough practices in scientific computing.** *PLoS Comput Biol*. 2017; **13**(6): e1005510.
[PubMed Abstract](#) | [Publisher Full Text](#) | [Free Full Text](#)
44. Li K, Lin X, Greenberg J: **Software Citation, Reuse and Metadata Considerations: An Exploratory Study Examining LAMMPS.** *Proc Assoc Inform Sci Tech*. 2016; **53**(1): 1–10.
[Publisher Full Text](#)
45. Kuchinke W, Ohmann C, Stenzhorn H, et al.: **Ensuring sustainability of software tools and services by cooperation with a research infrastructure.** *Per Med*. 2016; **13**(1): 43–55.
[PubMed Abstract](#) | [Publisher Full Text](#)
46. Loewe A, Seemann G, Wülfers EM, et al.: **SuLMASS - Sustainable Lifecycle Management for Scientific Software.** E-Science-Tage 2019: Data to Knowledge. 2019.
[Publisher Full Text](#)
47. Druskat S, Krause T, Lüdeling A, et al.: **Infrastrukturstrategien für nachhaltige Forschungssoftware in befristeten Projekten.** deRSE19 - Conference for Research Software Engineers in Germany. Potsdam, Germany. 2019.
[Publisher Full Text](#)
48. Morin A, Urban J, Sliz P: **A Quick Guide to Software Licensing for the Scientist-Programmer.** *PLoS Comput Biol*. 2012; **8**(7): e1002598.
[PubMed Abstract](#) | [Publisher Full Text](#) | [Free Full Text](#)
49. Katz DS, Ramnath R: **Looking at Software Sustainability and Productivity Challenges from NSF.** 2015. arXiv: 1508.03348.
[Reference Source](#)
50. DFG: **Nachhaltigkeit von Forschungssoftware.** 2016.
[Reference Source](#)
51. DFG: **Qualitätssicherung von Forschungssoftware durch ihre nachhaltige Nutzbarmachung.** 2019.
[Reference Source](#)
52. Chan Zuckerberg Initiative: **Essential Open Source Software for Science.** [Reference Source](#)
53. Rodríguez-Sánchez F, Marwick B, Lazowska E, et al.: **Academia's failure to retain data scientists.** *Science*. 2017; **355**(6323): 357–358.
[PubMed Abstract](#) | [Publisher Full Text](#)
54. Katz DS, Druskat S, Haines R, et al.: **The State of Sustainable Research Software: Learning from the Workshop on Sustainable Software for Science: Practice and Experiences (WSSSPE5.1).** *J Open Res Softw*. 2019; **7**(1): 11.
[Publisher Full Text](#)
55. Druskat S, Katz DS: **Mapping the Research Software Sustainability Space.** In: *2018 IEEE 14th International Conference on E-Science (e-Science)*. 2018; 25–30.
[Publisher Full Text](#)
56. Ye Y, Boyce RD, Davis MK, et al.: **Open Source Software Sustainability Models: Initial White Paper from the Informatics Technology for Cancer Research Sustainability and Industry Partnership Work Group.** 2019.
[Reference Source](#)
57. Hong NC: **Minimal information for reusable scientific software.** In: *2nd Workshop on Sustainable Software for Science: Practice and Experiences (WSSSPE2)*. 2014.
[Publisher Full Text](#)
58. Hasselbring W, Carr L, Hettrick S, et al.: **From FAIR research data toward FAIR and open research software.** *it - Information Technology*. 2020; **62**(1): 39–47.
[Publisher Full Text](#)
59. <https://www.acm.org/publications/policies/artifact-review-badging>.
60. Schlauch T, Meinel M, Haupt C: **DLR Software Engineering Guidelines.** Deutsches Zentrum für Luft- und Raumfahrt (DLR). 2018.
[Publisher Full Text](#)
61. Gomez-Diaz T, Recio T: **On the evaluation of research software: the CDUR procedure [version 2; peer review: 2 approved].** *F1000Res*. 2019; **8**: 1353.
[PubMed Abstract](#) | [Publisher Full Text](#) | [Free Full Text](#)
62. Fehr J, Heiland J, Himpe C, et al.: **Best practices for replicability, reproducibility and reusability of computer-based experiments exemplified by model reduction software.** *AIMS Mathematics*. 2016; **1**(3): 261–281.
[Publisher Full Text](#)
63. Jiménez RC, Kuzak M, Alhamdoosh M, et al.: **Four simple recommendations to encourage best practices in research software [version 1; peer review: 3 approved].** *F1000Res*. 2017; **6**: pii: ELIXIR-876.
[PubMed Abstract](#) | [Publisher Full Text](#) | [Free Full Text](#)
64. Hsu L, Hutchison VB, Langseth ML: **Measuring sustainability of seed-funded earth science informatics projects.** *PLoS One*. 2019; **14**(10): e0222807.
[PubMed Abstract](#) | [Publisher Full Text](#) | [Free Full Text](#)
65. Jackson M, Crouch S, Baxter R: **Software Evaluation Guide.** 2019.
[Reference Source](#)
66. rOpenSci; Anderson B, Chamberlain S, et al.: **Software Peer Review, Why? What? In: rOpenSci Packages: Development, Maintenance, and Peer Review Zenodo.** 2019.
[Publisher Full Text](#)
67. Lee BD: **Ten simple rules for documenting scientific software.** *PLoS Comput Biol*. 2018; **14**(12): e1006561.
[PubMed Abstract](#) | [Publisher Full Text](#) | [Free Full Text](#)
68. Venters CC, Capilla R, Betz S, et al.: **Software sustainability: Research and practice from a software architecture viewpoint.** *J Syst Software*. 2018; **138**: 174–188.
[Publisher Full Text](#)
69. SSI: **Writing and using a software management plan.** 2019.
[Reference Source](#)
70. Johanson A, Hasselbring W: **Software engineering for computational science: Past, present, future.** *Comput Sci Eng*. 2018; **20**(2): 90–109.
[Publisher Full Text](#)
71. Akhmerov A, Cruz M, Drost N, et al.: **Making Research Software a First-Class**

- Citizen in Research.** 2019.
[Publisher Full Text](#)
72. Hong NC: **Making Software A First-Class Citizen.** 2019.
[Publisher Full Text](#)
 73. Vereinigung der Kanzlerinnen und Kanzler der Universitäten Deutschlands: **Bayreuther Erklärung zu befristeten Beschäftigungsverhältnissen mit wissenschaftlichem und künstlerischem Personal in Universitäten.** 2019.
[Reference Source](#)
 74. Katz DS, McInnes LC, Bernholdt DE, *et al.*: **Community Organizations: Changing the Culture in Which Research Software Is Developed and Sustained.** *Comput Sci Eng.* 2019; **21**(2): 8–24.
[Publisher Full Text](#)
 75. Science Guide: **Room for everyone's talent.** 2019.
[Reference Source](#)
 76. Carver JC, Hong NPC, Thiruvathukal GK: **Software engineering for science.** CRC Press, 2016; 274.
[Reference Source](#)
 77. Jaffaldano G, Steinmacher I, Calefato F, *et al.*: **Why do developers take breaks from contributing to OSS projects? A preliminary analysis.** *arXiv: 1903.09528.* 2019.
[Reference Source](#)
 78. Allen A, Aragon C, Becker C, *et al.*: **Engineering Academic Software (Dagstuhl Perspectives Workshop 16252).** *Dagstuhl Manifestos* 2017; **6**(1): 1–20.
[Reference Source](#)
 79. Bundesministerium der Justiz und für Verbraucherschutz: **Gesetz über befristete Arbeitsverträge in der Wissenschaft.** 2017.
[Reference Source](#)
 80. Chang V, Mills H, Newhouse S: **From Open Source to long-term sustainability: Review of Business Models and Case studies.** In: Proceedings of the UK e-Science All Hands Meeting 2007 University of Edinburgh/University of Glasgow (acting through the NeSC) 2007.
[Reference Source](#)
 81. Aartsen W, Peeters P, Wagers S, *et al.*: **Getting Digital Assets from Public-Private Partnership Research Projects through "The Valley of Death," and Making Them Sustainable.** *Front Med (Lausanne).* 2018; **5**: 65.
[PubMed Abstract](#) | [Publisher Full Text](#) | [Free Full Text](#)
 82. Gabella C, Durinx C, Appel R: **Funding knowledgebases: Towards a sustainable funding model for the UniProt use case [version 2; peer review: 3 approved].** *F1000Res.* 2018; **6**: pii: ELIXIR-2051.
[PubMed Abstract](#) | [Publisher Full Text](#) | [Free Full Text](#)
 83. Ison J, Rapacki K, Ménager H, *et al.*: **Tools and data services registry: a community effort to document bioinformatics resources.** *Nucleic Acids Res.* 2016; **44**(D1): D38–D47.
[PubMed Abstract](#) | [Publisher Full Text](#) | [Free Full Text](#)
 84. Struck A: **Research Software Discovery: An Overview.** In: *2018 IEEE 14th International Conference on e-Science IEEE.* 2018.
[Publisher Full Text](#)
 85. Druskat S, Spaaks JH, Chue Hong N, *et al.*: **Citation File Format (CFF) - Specifications.** 2019.
[Publisher Full Text](#)
 86. **re3data.org – Registry of Research Data Repositories.**
[Publisher Full Text](#)
 87. Mönnich MW: **KVK - a meta catalog of libraries.** *LIBER Quarterly.* 2001; **11**(2): 121–127.
[Publisher Full Text](#)
 88. Kunze J, Scancelli J, Adams C, *et al.*: **The bagIt file packaging format (v1. 0).** RFC Editor. 2018; 8493.
 89. Bundesministerium der Justiz und für Verbraucherschutz, § 69a subsection (1) UrhG. 2014.
[Reference Source](#)
 90. Bundesministerium der Justiz und für Verbraucherschutz, § 69b UrhG. 2014.
[Reference Source](#)
 91. Struck A, Loewe A, Achhammer E, *et al.*: **A Guide for Publishing, Using, and Licensing Research Software in Germany.** *Zenodo.* 2020.
<http://www.doi.org/10.5281/zenodo.4327147>

Open Peer Review

Current Peer Review Status:  

Version 2

Reviewer Report 28 January 2021

<https://doi.org/10.5256/f1000research.31504.r78211>

© 2021 Hasselbring W. This is an open access peer review report distributed under the terms of the [Creative Commons Attribution License](#), which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited.



Wilhelm Hasselbring 

Software Engineering Group, Kiel University, Kiel, Germany

Thanks for delivering this revised version of your opinion article. I highly appreciate that you addressed all the concerns I had with the previous version, such that I can now fully approve your paper!

Competing Interests: No competing interests were disclosed.

Reviewer Expertise: Software Engineering

I confirm that I have read this submission and believe that I have an appropriate level of expertise to confirm that it is of an acceptable scientific standard.

Version 1

Reviewer Report 01 June 2020

<https://doi.org/10.5256/f1000research.25640.r62872>

© 2020 Bast R. This is an open access peer review report distributed under the terms of the [Creative Commons Attribution License](#), which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited.



Radovan Bast 

¹ Department of Information Technology, UiT The Arctic University of Norway, Tromsø, Norway

² Department of Information Technology, UiT The Arctic University of Norway, Tromsø, Norway

In "An environment for sustainable research software in Germany and beyond: current state, open

challenges, and call for action" the authors identify challenges for research software sustainability in Germany and beyond.

They examine the current state of research software sustainability and challenges in motivating sustainable research software development, selection criteria for funding, personnel, funding, infrastructure, and legal aspects, and offer recommendations for addressing these challenges. These sections are accompanied and with a story using a fictional character Kim which helps to relate these aspects to typical career stages of a research software engineer.

The article is thoroughly researched, well-written, and offers an excellent overview of the challenges when building an environment for sustainable research software. Most of the discussed challenges and recommendations carry beyond Germany and are relevant and transferable to other countries.

Below I give few (minor) suggestions for consideration when improving the manuscript.

Regarding the list of challenges under "Why sustainable research software in the first place?" (pages 5 and 6):

- Infrastructure issues: One design choice that often limits the use or usability of local infrastructure resources is that they are often bound to institutional user accounts and thus limit collaboration possibilities with collaborators in other institutions and countries. On the other hand, pooling of infrastructure resources which could enable collaboration across organizations can be limited by lack of authentication and authorization infrastructure (AAI) or legal constraints. Later in the paper the authors indeed mention AAI (page 12) but this could already be pointed out and connected earlier.
- Legal issues: Not only licensing is an issue but legal constraints or uncertainty about legal boundaries and identity federation can also limit the deployment of infrastructure services. Often the deployment and operation of infrastructure services is given to technical teams who may lack the legal support or expertise to clarify legal and privacy terms for the storage of data and processing of data.
- Funding issues: The challenge is not only that funding is scarce but also that it does not align well with pricing models of cloud infrastructure providers. It can be easier for research groups to spend a larger chunk of the budget towards the end of a year for hardware compared to pay possibly relatively modest monthly fees for a cloud service, which however may not fit into the budget forms. These budget constraints may also limit the possibility of pooling resources and sharing them with other research groups. Software cloud infrastructure is often not considered at all in the proposal. There is also a resistance among some of my research colleagues to pay 20-50 USD/ month for an infrastructure service which is sometimes solved by reinventing the service locally "for free".
- Another mismatch between traditional funding models and support of software which "must continue to exist" to be sustainable (page 5), is the experience that it can take months or years until the software is picked up by other groups and contributions and questions start to roll in. But by that time the funding of the project stopped, the developer (team) may have already moved on to other positions and projects, and may not have the time to react and help, even though they still may have interest and the knowledge. Our traditional

funding models consider the software to be "done" by the end of the project.

Selection criteria for "How to decide which software to sustain?" (page 9):

- The authors mention "usage and impact", "software quality", as well as "maturity". But I would like to see also "openness and transparency" among these. The reason is that we can expect the research community to adapt to these or any metrics and we will over time observe what we measure. Any set of metrics could be criticized as to some extent being arbitrary but the advantage of including "openness and transparency" is that the community as whole would benefit from such a metric [Enrico Glerean, "Responsible conduct of research and questionable research practices", presentation, slide 47¹].

Regarding "Who sustains research software?":

- The authors discuss the lack of recognition and awareness, as well as lack of career opportunities. It is also about respect and I was happy to see the sentence: "Not all researchers that think of themselves as RSEs pursue a faculty position as their main career goal." I have experienced that RSEs are sometimes regarded as those who somehow "failed" to obtain a faculty position whereas many RSEs have chosen this position over a faculty position because it was a better fit for their career goals. This misunderstanding can lead to a lack of respect towards this position and this career choice and can lead to excellent personnel leaving the academic environment towards commercial employment, possibly not primarily for financial reasons but sometimes to be more respected and recognized.

Archiving and software preservation (page 13):

- The authors mention Docker but also Singularity should be mentioned as a tool since it is getting traction in particular on many-user systems such as higher performance computing clusters.

Legal aspects (page 14):

- Re-licensing is mentioned in the story box and the text starts by pointing out that licensing is often not considered early enough in the project. Indeed re-licensing later in the project can be not only legally, but also organizationally very tricky, in particular for projects which developed over many years and involved many contributors in different organizations. This could be pointed out in the text as additional motivation to consider these very early in the project.
- I very much like the recommendation of providing a legal help desk for research groups to avoid the problem that out of uncertainty and fear of making a legal mistake some research groups end up not choosing any license at all which may limit further reuse of the software.
- The manuscript presents a decision tree for contributors (Figure 1) and also discusses contributor license agreements. It could be useful to point out that without clear policies or legal help desks, individuals or organizations may be hesitant to contribute to a project because they may not feel confident having enough knowledge or authority to sign such agreements and too many legal steps and question can also raise the barrier to contribute, in particular for smaller projects. Also here clear guidelines and a support desk can help removing these barriers.

References

1. Enrico Glerean: Responsible conduct of research and questionable research practices. 2018.

[Reference Source](#)

Is the topic of the opinion article discussed accurately in the context of the current literature?

Yes

Are all factual statements correct and adequately supported by citations?

Yes

Are arguments sufficiently supported by evidence from the published literature?

Yes

Are the conclusions drawn balanced and justified on the basis of the presented arguments?

Yes

Competing Interests: No competing interests were disclosed.

Reviewer Expertise: computational chemistry, research software engineering

I confirm that I have read this submission and believe that I have an appropriate level of expertise to confirm that it is of an acceptable scientific standard.

Author Response 17 Dec 2020

Axel Loewe

We thank you for the thorough review and constructive feedback regarding the manuscript. Below, we address the issues raised by you point-by-point. Our responses are set in italics.

Regarding the list of challenges under "Why sustainable research software in the first place?" (pages 5 and 6):

Infrastructure issues: One design choice that often limits the use or usability of local infrastructure resources is that they are often bound to institutional user accounts and thus limit collaboration possibilities with collaborators in other institutions and countries. On the other hand, pooling of infrastructure resources which could enable collaboration across organizations can be limited by lack of authentication and authorization infrastructure (AAI) or legal constraints. Later in the paper the authors indeed mention AAI (page 12) but this could already be pointed out and connected earlier.

We thank the reviewer for their suggestion, and have included the mentioned issues in the respective list in the section "Why sustainable research software in the first place?".

Legal issues: Not only licensing is an issue but legal constraints or uncertainty about legal boundaries and identity federation can also limit the deployment of infrastructure services. Often the deployment and operation of infrastructure services is given to technical teams who may lack the legal support or expertise to clarify legal and privacy terms for the storage of data and processing of data.

We thank the reviewer for their suggestion, and have included the mentioned issues in the respective list in the section "Why sustainable research software in the first place?".

Funding issues: The challenge is not only that funding is scarce but also that it does not align well with pricing models of cloud infrastructure providers. It can be easier for research groups to spend a larger chunk of the budget towards the end of a year for hardware compared to pay possibly relatively modest monthly fees for a cloud service, which however may not fit into the budget forms. These budget constraints may also limit the possibility of pooling resources and sharing them with other research groups. Software cloud infrastructure is often not considered at all in the proposal. There is also a resistance among some of my research colleagues to pay 20-50 USD/ month for an infrastructure service which is sometimes solved by reinventing the service locally "for free".

Another mismatch between traditional funding models and support of software which "must continue to exist" to be sustainable (page 5), is the experience that it can take months or years until the software is picked up by other groups and contributions and questions start to roll in. But by that time the funding of the project stopped, the developer (team) may have already moved on to other positions and projects, and may not have the time to react and help, even though they still may have interest and the knowledge. Our traditional funding models consider the software to be "done" by the end of the project.

We thank the reviewer for these two comments. To address them, we have extended the "Funding issues" list item with a discussion of these issues.

Selection criteria for "How to decide which software to sustain?" (page 9):

The authors mention "usage and impact", "software quality", as well as "maturity". But I would like to see also "openness and transparency" among these. The reason is that we can expect the research community to adapt to these or any metrics and we will over time observe what we measure. Any set of metrics could be criticized as to some extent being arbitrary but the advantage of including "openness and transparency" is that the community as whole would benefit from such a metric [Enrico Glerean, "Responsible conduct of research and questionable research practices", presentation, slide 471].

We thank the reviewer for this suggestion and realized that indeed most aspects in this section (6-10, 12, 13) are actually related to openness and transparency. Therefore, we changed the title of this section to "Software transparency and quality".

Regarding "Who sustains research software?":

The authors discuss the lack of recognition and awareness, as well as lack of career opportunities. It is also about respect and I was happy to see the sentence: "Not all researchers that think of themselves as RSEs pursue a faculty position as their main career goal." I have experienced that RSEs are sometimes regarded as those who somehow "failed" to obtain a faculty position whereas many RSEs have chosen this position over a faculty position because it was a better fit for their career goals. This misunderstanding can lead to a lack of respect towards this position and this career choice and can lead to excellent personnel leaving the academic environment towards commercial employment, possibly not primarily for financial reasons but sometimes to be more respected and recognized.

We fully agree with the reviewer and thank them for the renewed confirmation that this is seen as problematic not only by the authors.

Archiving and software preservation (page 13):

The authors mention Docker but also Singularity should be mentioned as a tool since it is getting traction in particular on many-user systems such as higher performance computing clusters.

We thank the reviewer for this suggestion and now also mention Singularity and GUIX. However, we are not aiming for an exhaustive list, as options change dynamically and might even be specific to certain research communities.

Legal aspects (page 14):

Re-licensing is mentioned in the story box and the text starts by pointing out that licensing is often not considered early enough in the project. Indeed re-licensing later in the project can be not only legally, but also organizationally very tricky, in particular for projects which developed over many years and involved many contributors in different organizations. This could be pointed out in the text as additional motivation to consider these very early in the project.

We thank the reviewer to point this out and it also nicely fits into the message of increasing the awareness of legal aspects early on in the project. We have added the suggested sentence in the manuscript.

I very much like the recommendation of providing a legal help desk for research groups to avoid the problem that out of uncertainty and fear of making a legal mistake some research groups end up not choosing any license at all which may limit further reuse of the software. *Thank you for supporting this idea. We have further included your idea of avoiding any license out of a fear to make legal mistakes.*

The manuscript presents a decision tree for contributors (Figure 1) and also discusses contributor license agreements. It could be useful to point out that without clear policies or legal help desks, individuals or organizations may be hesitant to contribute to a project because they may not feel confident having enough knowledge or authority to sign such agreements and too many legal steps and question can also raise the barrier to contribute, in particular for smaller projects. Also here clear guidelines and a support desk can help removing these barriers.

We have decided to take the decision trees out of the manuscript to strengthen our point of publishing software under a FLOSS license. Instead, we published the decision trees together with documentation templates under a Creative Commons license via Zenodo:

<https://zenodo.org/record/4327148#.X9n6ui337OQ>. In order to strengthen the point you addressed, we added some more details related to infrastructural investment.

Competing Interests: No competing interests were disclosed.

Reviewer Report 13 May 2020

<https://doi.org/10.5256/f1000research.25640.r62873>

© 2020 Hasselbring W. This is an open access peer review report distributed under the terms of the [Creative Commons Attribution License](#), which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited.



Wilhelm Hasselbring 

¹ Software Engineering Group, Kiel University, Kiel, Germany

² Software Engineering Group, Kiel University, Kiel, Germany

The authors describe the state of the practice and current challenges for research software sustainability and suggest measures towards improvements that can solve these challenges. In particular, they propose to fund a German Software Sustainability Institute. The paper is the result of a community effort, with work undertaken during two workshops and subsequent collaborative work across the larger RSE community in Germany.

The UK Software Sustainability Institute has already been established during a decade (<https://www.software.ac.uk/blog/2020-05-05-impact-institute-10-years>). Thus, the idea of such an institute is not new, but it makes sense to take a specific look at the German situation. Besides universities, the German states (local and in particular federal) fund significant large-scale research associations (Helmholtz/DLR, Max-Planck, Leibniz). This is not the case for most other European states, at least not with a similar scale. Another specialty is the lack of long-term funding for research software engineers, as discussed by the authors.

The paper is well-written and easy to read. I like the boxed story of Kim's career path.

However, I've some suggestions for improving the paper:

- Concerning the statement "In order to support research, a sustainable software must be correct", I suggest to include a short discussion of the test oracle problem for scientific software (see for instance <https://doi.org/10.1109/SECSE.2013.6615099>¹).
- Concerning the discussion of "The list of criteria presented in this section could be the basis for a structured review process..." I suggest to include two additional initiatives for software review. The first is artifact evaluation in computer science conferences (the process is explained in <https://doi.org/10.1515/itit-2019-0040>²). The second is the SPEC Research Group's review process of tools for quantitative system evaluation and analysis (<https://research.spec.org/tools/submission.html>).

The authors write "We also argue that truly sustainable research software must ideally be published under a Free/Libre Open Source Software (FLOSS) license, and follow an open development model..." what I fully support (see for instance <https://doi.org/10.1515/itit-2019-0040>²). However, later under the section heading "Legal aspects" this requirement is thwarted. I fully agree that legal aspects have to be considered, but the general bias of this section seems to be on commercial licensing of research software. For instance, the decision tree in Figure 1 starts with the question "Licensing planned?". I assume that commercial licensing is meant, but this is not clear since the figures are not explained in the paper. Instead, the process should start with open sourcing the software. If licenses such as Apache or MIT are applied, the research institutions may later still commercialize the software if appropriate. Such open source licensing is also beneficial for start-ups, that intend to provide professional services for the software.

My experience with technology transfer units of German universities and research institutes is that they do not understand the ideas of open source business models (see for instance <https://doi.org/10.1109/MC.2019.2898163>³). Their focus is on patents and commercializing licenses, sometimes also on start-ups. Conversely, in the software industry, one major motivation for open sourcing software is on improving the quality of software. I cite from

<https://doi.org/10.1109/ICSAW.2017.114> : “the open-source approach has some psychological effects: Developers show a tendency to apply higher quality standards if they know that the code will be publicly available.” For sustainability, quality is an important property of software.

The Figures 1-4 do more harm than good. They are daunting to researchers who intend to publish their code open source. These figures should be removed from the paper, they are useless without proper explanation.

I suggest that the authors focus in the present paper on their main message (request for funding a German Software Sustainability Institute, which I fully support). Figures 1-4 could be moved to a separate paper, enriched with proper explanation.

References

1. Kanewala U, Bieman JM: Techniques for testing scientific programs without an oracle. *IEEE Xplore*. 2013. [Publisher Full Text](#)
2. Hasselbring W, Carr L, Hettrick S, Packer H, et al.: From FAIR research data toward FAIR and open research software. *it - Information Technology*. 2020; **62** (1): 39-47 [Publisher Full Text](#)
3. Riehle D: The Innovations of Open Source. *Computer*. 2019; **52** (4): 59-63 [Publisher Full Text](#)
4. Hasselbring W, Steinacker G: Microservice Architectures for Scalability, Agility and Reliability in E-Commerce. *IEEE*. 2017. [Publisher Full Text](#)
5. Tools Submission Portal. *SPEC Research Group*. [Reference Source](#)

Is the topic of the opinion article discussed accurately in the context of the current literature?

Yes

Are all factual statements correct and adequately supported by citations?

Yes

Are arguments sufficiently supported by evidence from the published literature?

Partly

Are the conclusions drawn balanced and justified on the basis of the presented arguments?

Partly

Competing Interests: No competing interests were disclosed.

Reviewer Expertise: Software Engineering

I confirm that I have read this submission and believe that I have an appropriate level of expertise to confirm that it is of an acceptable scientific standard, however I have significant reservations, as outlined above.

Author Response 17 Dec 2020

Axel Loewe

Thank you for the thorough review and constructive feedback regarding our manuscript. Below, we address the issues raised by you point-by-point. Our responses are set in italics.

Concerning the statement “In order to support research, a sustainable software must be correct”, I suggest to include a short discussion of the test oracle problem for scientific software (see for instance <https://doi.org/10.1109/SECSE.2013.66150991>).

We thank the reviewer for this suggestion, and included a brief discussion of the test oracle problem as suggested, and additionally of further challenges to verification and validation, such as large configuration spaces and heterogeneous data (as discussed in e.g.

<https://doi.org/10.1109/SE4Science.2019.00010>), and have suggested to implement the solutions mentioned in the literature.

Concerning the discussion of “The list of criteria presented in this section could be the basis for a structured review process...” I suggest to include two additional initiatives for software review. The first is artifact evaluation in computer science conferences (the process is explained in <https://doi.org/10.1515/itit-2019-00402>). The second is the SPEC Research Group’s review process of tools for quantitative system evaluation and analysis (<https://research.spec.org/tools/submission.html>).

We thank the reviewer for this suggestion and included the artifact review approach in the introduction to the criteria section. The aspects of repeatability, reproducibility, and replicability are aimed more at the results of computational research rather than research software itself, we feel. Therefore, we didn’t include specific criteria in the list suggested to be used when evaluating research software for sustained funding.

While the SPEC submission process is very clear, we could not find any concrete criteria applied during the review (except for requirements regarding the license).

The authors write “We also argue that truly sustainable research software must ideally be published under a Free/Libre Open Source Software (FLOSS) license, and follow an open development model...” what I fully support (see for instance <https://doi.org/10.1515/itit-2019-00402>). However, later under the section heading “Legal aspects” this requirement is thwarted. I fully agree that legal aspects have to be considered, but the general bias of this section seems to be on commercial licensing of research software. For instance, the decision tree in Figure 1 starts with the question “Licensing planned?”. I assume that commercial licensing is meant, but this is not clear since the figures are not explained in the paper. Instead, the process should start with open sourcing the software. If licenses such as Apache or MIT are applied, the research institutions may later still commercialize the software if appropriate. Such open source licensing is also beneficial for start-ups that intend to provide professional services for the software.

My experience with technology transfer units of German universities and research institutes is that they do not understand the ideas of open source business models (see for instance <https://doi.org/10.1109/MC.2019.28981633>). Their focus is on patents and commercializing licenses, sometimes also on start-ups. Conversely, in the software industry, one major motivation for open sourcing software is on improving the quality of software. I cite from <https://doi.org/10.1109/ICSAW.2017.114> : “the open-source approach has some psychological effects: Developers show a tendency to apply higher quality standards if they know that the code will be publicly available.” For sustainability, quality is an important property of software.

As further detailed below, we have moved the decision trees out of this manuscript as we see the problems and agree to your arguments. We like your suggested aspect of commercialization of FLOSS licensed software and included this aspect in the manuscript.

The Figures 1-4 do more harm than good. They are daunting to researchers who intend to publish their code open source. These figures should be removed from the paper, they are useless without proper explanation.

I suggest that the authors focus in the present paper on their main message (request for funding a German Software Sustainability Institute, which I fully support). Figures 1-4 could be moved to a separate paper, enriched with proper explanation.

We thank the reviewer for this suggestion. Our initial thought was to place these decision trees in the supplemental material but did not realize that this is not the policy of f1000. The editorial team moved them into the main article. This is the reason why the decision trees appeared without additional information in the manuscript. We have now decided to take the Figures out and have published them together with documentation templates in a separate report via Zenodo unde a Creative Commons license, see

<https://zenodo.org/record/4327148#.X9n6ui337OQ>. This report is now cited in the F1000 manuscript.

Competing Interests: No competing interests were disclosed.

Comments on this article

Version 2

Reader Comment 22 Feb 2021

Dirk Feuchter, Karlsruhe Institute of Technology, Karlsruhe, Germany

Dear Mr. Anzt, dear Mr. Loewe, dear Mr. Bach, dear Mr. Seemann, as well as dear other authors and reviewers,

just discovered your feedback to my Comment. Thank you very much for that.

I can well understand that your community has in mind proprietary licensing at most in exceptional situations.

Please note and obviously you keep this already in mind, that even in the case of pure F(L)OSS licensing, i.e. completely without proprietary licensing intentions the topic of "F(L)OSS-right compliance" should be considered especially in the sense of sustainability of the software.

Hence, and only as a suggestion, the „OpenChain movement“ and the new ISO 5230 standard resulting from it at the end of 2020 should possibly be mentioned and cited in your paper; possibly in „Legal Aspects -> Challenges and clarifications“. after your sentence „...The challenge in the use,

distribution, and commercialization of software is to determine the chain of rights and to identify all right holders...."

Best regards

Dirk Feuchter

Competing Interests: No competing interests were disclosed.

Version 1

Author Response 17 Dec 2020

Axel Loewe

We thank you for your thorough review and constructive feedback regarding the manuscript. Below, we address the issues raised by you point-by-point. Our responses are set in italics.

Dear Mr. Anzt, dear Mr. Loewe, dear Mr. Bach, dear Mr. Seemann, dear Elke, and dear Sven, as well as dear authors as yet unknown to me,

I am working at KIT in the field of Innovation and Relations Management, especially Licensing of Intellectual Property Rights from KIT to Free Market Economy. My particular focus is to Out-license Computerprograms to Third Parties and Industry.

Thank you for your great efforts with your extensive and interesting article including the nicely written and ever-recurring story of Kim.

Your FLOSS-based approach for sustainable software development is holding immense savings potential. That's great.

Following are my comments to your article from the perspective of a TTO license manager, typically supporting RSEs in cases of proprietary licensing e.g. to spin-offs or industrial companies.

Hence, most of my comments might go in a slightly different direction than the main focus of your article, but in my point of view these comments are complementary. Hence, I wonder what you think of them. Looking forward to your opinion and your feedback to my comments, if you like also gladly by telephone

Best regards

Dirk Feuchter

Thank you very much for your comments regarding distribution issues, IP compliance and commercialization. We have generally subsumed these issues under the more general term "availability". We believe that this implies that software is legally available only if it is licensed, either under an open source license or proprietarily. This impacts possible modes of distribution, but does not concern actual (commercial) distribution processes. As a community of Research Software Engineers, we strongly believe that publicly funded research software should be F(L)OSS-licensed per default, although we recognize that this may not always be possible (and hence have weakened "must" to "should" as you suggested in the respective paragraph under "Why sustainable research software in the first place"). We have further avoided any changes to the text that would weaken this point, as we see no central obstacles to making publicly funded research software open source in general. Contrarily, we do not accept that IP should

override public interest (both intellectually and fiscally) here, some corner cases excluded. Concurrently, we purposefully do not focus on marketability and commercialization of research software. Instead, we see commercial opportunities, e.g., in the provision of services for a research software product, whereas the product itself should remain free and open source.

Thank you, also, for notifying us of some errors in the text itself and the figures, which we are fixing in the next version.

Below, we address some of your concrete suggestions in more detail:

Abstract:

I would like to suggest to extend

„Research software must be sustainable in order to understand, replicate, reproduce, and...”

as follows

„Research software must be sustainable in order to understand, replicate, reproduce, distribute and...”

We thank you for the comment. The development of research software does not focus on distribution of software results, which is more of a business aspect. The availability, regardless of the actual distribution, is noted in the next sentence of the abstract.

Abstract:

I would like to suggest to extend

„In other words, software must be available, discoverable, usable, and adaptable to new needs, both now and in the future.”

as follows

„In other words, software must be available, (IP-/FLOSS)compliant*, discoverable, usable, and adaptable to new needs, both now and in the future.”

*By „(IP-/FLOSS)compliant” I mean

in compliance

with intellectual property of third-party suppliers,

with the terms of free/libre open source licenses and with the aim to protect own intellectual property from unintended disclosure

We thank you for the comment. As a community of RSEs we are aiming for FLOSS whenever feasible but acknowledge that there are scenarios in which non-FLOSS licenses need to be considered.

Suggestion of a new penultimate paragraph in subchapter „Stakeholder motivations for research software sustainability” as follows: RSEs, Research leaders and research performing organisations are interested in software sustainability also in the sense that their (research)software is sustainable concerning legal compliance. This is an important issue distributing (research)software for both academic purposes as well as commercial purposes. For the latter RSEs and their research leaders (typically contacting their TTO) are interested to market (parts of) their research software and/or additional connectable closed software (which they prevent from unintended disclosure) to a spin-off or an industrial company using a proprietary or a dual licensing model. For both, academic and commercial purposes RSEs and their research leaders and their research performing organisation are interested that their (research) software plus any connectable closed software is compliant with intellectual property of third-party suppliers and compliant with the terms of free/libre open source licenses. An important stakeholder motivation is therefore „software

sustainability with the aim of clarification of all software rights ownerships".

We thank you for the suggestion. While we aim for reusable software in terms of licenses we as a community do not focus on commercial purposes.

Please replace Subchapter heading „Challenges and clarifications Clarification of rights“ by „Challenges and Clarification of rights“

We thank you for pointing this out and corrected the structure (section and subsection headings, respectively).

The term "research institution" appears in the glossary as a duplicate. Therefore, "research institution" should be deleted in the brackets.

Thank you, we have replaced it by "national labs".

Competing Interests: No competing interests were disclosed.

Reader Comment 19 May 2020

Dirk Feuchter, Karlsruhe Institute of Technology (KIT), Eggenstein-Leopoldshafen and Karlsruhe, Germany

Subject: Feedback to your article „An environment for sustainable research software in Germany and beyond: current state, open challenges, and call for action...“.

Dear Mr. Anzt, dear Mr. Loewe, dear Mr. Bach, dear Mr. Seemann, dear Elke, and dear Sven, as well as dear authors as yet unknown to me,

I am working at KIT in the field of Innovation and Relations Management, especially Licensing of Intellectual Property Rights from KIT to Free Market Economy. My particular focus is to Out-license Computerprograms to Third Parties and Industry.

Thank you for your great efforts with your extensive and interesting article including the nicely written and ever-recurring story of Kim.

Your FLOSS-based approach for sustainable software development is holding immense savings potential. That's great.

Following are my comments to your article from the perspective of a TTO license manager, typically supporting RSEs in cases of proprietary licensing e.g. to spin-offs or industrial companies.

Hence, most of my comments might go in a slightly different direction than the main focus of your article, but in my point of view these comments are complementary. Hence, I wonder what you think of them. Looking forward to your opinion and your feedback to my comments, if you like also gladly by telephone

Best regards

Dirk Feuchter

#####

Comment-DF-01:

Abstract:

I would like to suggest to extend

„Research software must be sustainable in order to understand, replicate, reproduce, and...”

as follows

„Research software must be sustainable in order to understand, replicate, reproduce, **distribute** and...”

#####

Comment-DF-02:

Abstract:

I would like to suggest to extend

„In other words, software must be available, discoverable, usable, and adaptable to new needs, both now and in the future.”

as follows

„In other words, software must be available, (IP-/FLOSS)**compliant***, discoverable, usable, and adaptable to new needs, both now and in the future.”

*By „(IP-/FLOSS)**compliant**” I mean
in compliance

- with intellectual property of third-party suppliers,
- with the terms of free/libre open source licenses and with the aim to protect own intellectual property from unintended disclosure

#####Comment-DF-03:Why sustainable research software in the first place =>I would like to suggest to extend„In order to support research, a sustainable software must be correct¹⁴, validatable, understandable, documented, publicly released,...” as follows „In order to support research, a sustainable software must be correct¹⁴, (IP-/FLOSS) **compliant*** validatable, understandable, documented, publicly released,...” *Concerning „(IP-/FLOSS)**compliant**” please see upon #####

- Comment-DF-04:Why sustainable research software in the first place =>„We also argue that truly sustainable research software must ideally be published under a Free/Libre Open Source Software (FLOSS) license, and...”„We also argue that truly sustainable research software **should typically** be published under a Free/Libre Open Source Software (FLOSS) license, and...”

#####

Comment-DF-05:Why sustainable research software in the first place =>„Therefore, if research software is publicly funded, it should be freely available under a FLOSS license.”„Therefore, if research software is publicly funded, it should be **normally** freely available under a FLOSS license.
#####

Comment-DF-06:Why sustainable research software in the first place =>I would like to suggest to extend„Legal issues: Many obstacles for research software pertain to legal issues, such as

applicable licensing and compatibility of licenses⁴⁵, and decisions about license types. as follows „Legal issues: Many obstacles for research software pertain to legal issues, such as **IT law, copyright law, copyright notices and author attributions**, applicable licensing and compatibility of licenses⁴⁵, and decisions about license types.

#####

Comment-DF-07:Why sustainable research software in the first place =>„A subset of this **group** may be interested in ...“.Which „group“ do you mean? => the „Taxpayers“? #####

Comment-DF-08:Suggestion of a new penultimate paragraph in subchapter „Stakeholder motivations for research software sustainability“ as follows: RSEs, Research leaders and research performing organisations are interested in software sustainability also in the sense that their (research)software is sustainable concerning legal compliance.This is an important issue distributing (research)software for both academic purposes as well as commercial purposes. For the latter RSEs and their research leaders (typically contacting their TTO) are interested to market (parts of) their research software and/or additional connectable closed software (which they prevent from unintended disclosure) to a spin-off or an industrial company using a proprietary or a dual licensing model. For both, academic and commercial purposes RSEs and their research leaders and their research performing organisation are interested that their (research) software plus any connectable closed software is compliant with intellectual property of third-party suppliers and compliant with the terms of free/libre open source licenses. An important stakeholder motivation is therefore „software sustainability with the aim of clarification of all software rights ownerships“.

#####

Comment-DF-09:Abstract:I would like to suggest to extend„Failing to do so will threaten the quality and validity of research..“ as follows „Failing to do so will threaten the quality, Xi and validity of research..“ // X1= marketability or X2= distribution // As TTO-license-manager I personally would prefer X1 but X2 is fine as well

#####Comment-DF-10:In the heading such as „Why sustainable research software in the first place?“, one might prefix a chapter number, just as announced at the end of the introductory background: „This all leads to unmet requirements and unsolved challenges that we want to highlight in this paper by elaborating on (1)...(2)...(3)...(4)...(5)...(6)...“Hence, instead of„Why sustainable research software in the first place?“use„(1) Why sustainable research software in the first place?“and so on:(2) How to decide which software to sustain?(3) Who sustains research software?(4) How can research software be sustainably funded?(5) Which infrastructure is needed to sustain research software?(6) Legal aspects.That's a matter of taste, of course.

#####

Comment-DF-11:Please replace Subchapter heading„Challenges and clarifications Clarification of rights“by „Challenges and Clarification of rights“

#####

Comment-DF-12:In both Fig. 1 and „Legal aspects/Challenges and clarifications of rights“instead of „Subject to directives“ {weisungsgebunden dt.} from UrhG 69 bI propose to write „in execution of his duties {in Wahrnehmung seiner Aufgaben dt.}“ from UrhG 69b, in no way to constrain scientists, RSEs and Research leaders, but in order to free up the scope for decision-making and thus open up opportunities.[Regardless of that, I would translate the German "weisungsgebunden" from „UrhG 69 b“ with "bound by instructions"(short) or with "following the instructions given by his employer"(long)]

#####

- Comment-DF-13:In Subchapter „Conclusions“I would not write the following sentence, or at most I would write it differently.„We encourage the research funding bodies to reflect the licensing models for academic software development, and to decide whether the “public money, public code” paradigm justifies the requirement that all publicly funded software has to be publicly available under a Free/Libre Open Source Software (FLOSS) license.“ I can understand your point of view and this sentence. But isn't there a lack of perspective regarding those research leaders or RSE-teams, who are considering a foundation based on a proprietary license model regarding their software development or RSE teams whose software developments are taken over and marketed by industrial companies in or outside of Germany, without return to the research performing organizations and the corresponding RSE teams.Hence, an alternative formulation might be as follows: „We encourage the research funding bodies to reflect the licensing models for academic software development, and to suggest research performing organisations and their research leaders in the sense of sustainability to make their software typically publicly available under a FLOSS-license but also to take into consideration revenue-oriented approaches such as FLOSS business models or proprietary licensing if applicable.“

In my opinion, the final decision **to license in and out as well as to transfer computer programs** under FLOSS licenses should be the responsibility of the (authorised for this purpose by the Presidium/Board of Directors of the research performing organisation) Research Leaders, with whom the computerprogram developing RSEs should therefore consult. Furthermore, the RSEs and their research leaders ideally should have the opportunity at their research institution to contact (preferably at an early stage) science-supporting specialist departments such as Legal, TTO, research software local task forces or even a SSI-like nationwide institution and seek advice.

#####

Comment-DF-14:In Fig.3 it should read "Check (1)..." instead of "Check (2)..." in both the centre left and the top right.

#####

Comment-DF-15: The term "research institution" appears in the glossary as a duplicate. Therefore, "research institution" should be deleted in the brackets.

#####

- Best regardsDirk Feuchter-----Karlsruhe Institute of Technology (KIT)INNOVATION AND RELATIONS MANAGEMENT (IRM)Intellectual Property

ManagementDr. Dirk Feuchter (Licenses)Hermann-von-Helmholtz-Platz 1D-76344 Eggenstein
LeopoldshafenGermanyPhone: +49 721 608-2-3921E-Mail: dirk.feuchter at kit.eduWeb:
https://www.irm.kit.edu/116_1500.php <https://www.irm.kit.edu/english/91.php>
www.irm.kit.edu/

Competing Interests: No competing interests were disclosed.

The benefits of publishing with F1000Research:

- Your article is published within days, with no editorial bias
- You can publish traditional articles, null/negative results, case reports, data notes and more
- The peer review process is transparent and collaborative
- Your article is indexed in PubMed after passing peer review
- Dedicated customer support at every stage

For pre-submission enquiries, contact research@f1000.com

F1000Research